

Supplementary Information

Learning Together, Better, Faster - On the Timescales of Learning Social Learning Strategies

Wolfram Barfuss

Paula Tittel

Richard P. Mann

April 17, 2026

Table of contents

1	Overview	3
2	Uncertain decision problem	5
2.1	Observation model	5
2.2	Environment classes	9
2.3	Single-agent environment	10
2.4	Two-agent environment	14
3	Learning Dynamics	19
3.1	Model-based learning dynamics	19
3.2	Model-free learning dynamics	29
3.3	Testing the implementation	29
4	Result Final rewards	32
5	Result Final strategies	39
6	Result Strategy convergence timescales	46
7	Result Reward performance timescales	52
8	Auxiliary Robustness checks	68
8.1	Learning rate α	69
8.2	Convergence threshold ε	72
	Appendices	75
A	Simulation scripts	75
A.1	Reward outperformance measure	75
A.2	Initial strategies	75
A.3	Learning trajectories	77
A.4	Varying noise	82
A.5	Noise variation comparison	87
B	Obtain noise variation data	89

1 Overview

Sections 2 and 3 define our model. Section 2 contains the implementation of our two decision environments: the single-agent and the two-agent uncertain decision problems. Section 3 contains the implementation of the model-based and model-free learning dynamics.

Sections 4 to 7 contain our results. Section 4 shows how we compute the final reward outperformance across different levels of environmental uncertainty. Section 5 shows how we visualize the final strategies across different levels of environmental uncertainty. Section [-@#sec-results-strategy-convergence-timescales] shows how we visualize the time steps required to reach the final strategies across different levels of environmental uncertainty. Finally, section 7 compute and visualizes the timescales required to reach almost the rewards of the final strategies across different levels of environmental uncertainty.

Appendices A and B contains the code to obtain the simulation data used in our results. Section A contains the scripts to run the simulations of our model. Section B contains the code to obtain the simulation data used in our results.

This supplementary information was created in a *fully reproducible writing and computing environment* with the help of [nbdev](#) and [quarto](#), version

```
!quarto --version
```

1.8.26

A Python environment with the following packages was used:

```
import sys
import importlib.metadata as md

packages = ["numpy", "matplotlib", "pandas", "scipy", "sympy", "scikit-optimize",
    ↪ "jax",
    "nbdev"]
print("Python version:"); print(sys.version); print()
print("Package versions:");
for pkg in packages: print(f"{pkg}: {md.version(pkg)}")
```

Python version:

3.14.2 | packaged by Anaconda, Inc. | (main, Dec 19 2025, 11:42:52) [Clang 20.1.8]

Package versions:

numpy: 2.3.5

matplotlib: 3.10.7

pandas: 2.3.3

scipy: 1.16.3

sympy: 1.14.0

```
scikit-optimize: 0.10.2  
jax: 0.8.2  
nbdev: 2.4.7
```

The journal template is available from [Quarto Extensions](#). We choose the `arxiv` template from [mikemahoney218](#), which must be installed into an existing Quarto project via

```
quarto add mikemahoney218/quarto-arxiv
```

to be able to render the main manuscript file in the arXiv format via

```
quarto render Article.ipynb
```

Finally, the supplementary information document is compiled as a Quarto book project and configured in the `_quarto.yml` file. To make the PDF output visually pleasing, Quarto's Code Visibility Extension is used to hide some code lines that only serve PDF aesthetics. The extension must be installed via

```
quarto install extension jjallaire/code-visibility
```

to be able to render the supplementary information to PDF via

```
quarto render .
```

2 Uncertain decision problem

Here we describe the model environment of the uncertain decision problems and provide two Python classes, one for the single-agent environment, the other for the two-agent environment. The interface to the learning dynamics is models according to the first author's Python package [pyCRLD](#). Note that we reimplement the relevant parts of pyCRLD in Chapter 3 to keep dependencies minimal.

```
#| default_exp UncertainDecisionProblem
```

```
#| export
import numpy as np
import itertools as it
from scipy.stats import norm
from fastcore.utils import patch
import matplotlib.pyplot as plt
np.set_printoptions(legacy='1.25')
```

The environment of the multiagent-environment interface is composed of two parts: the natural environment and the social environment.

The natural environment consists of two states, A and B. In each episode, the natural environment is in state A with probability p_A and in state B with probability $1 - p_A$.

Agent can choose between two actions, A and B. Depending on the nature state, agents receive a positive reward if they choose the action corresponding to the state of the natural environment and -1 otherwise.

2.1 Observation model

However, the agents cannot directly observe the true state of nature. They only receive a noisy signal which can take one of four values: A , a , b , B . Here,

- A denotes nature to be *very likely* in state A,
- a denotes nature to be *likely* in state A,
- b denotes nature to be *likely* in state B, and
- B denotes nature to be *very likely* in state B.

The probability of receiving each signal o , O^{so} , depends on the true state of nature s and the noiselevel σ . We model the probability of receiving each signal with a normal distribution $\mathcal{N}_{\mu,\sigma}$ centered around the true state of nature (suppose $\mu = 0$) and the other nature state one unit length apart (at 1). The noise levels σ corresponds to the standard deviation of the normal distribution. Agents make observation

- A with probability $\int_{-\infty}^{-d} \mathcal{N}_{0,\sigma}(x) dx$,

- a with probability $\int_{-\frac{d}{1+d}}^{0.5} \mathcal{N}_{0,\sigma}(x) dx$,
- b with probability $\int_{0.5}^{\frac{d}{1+d}} \mathcal{N}_{0,\sigma}(x) dx$, and
- B with probability $\int_{1+d}^{0.5\infty} \mathcal{N}_{0,\sigma}(x) dx$,

where d is the distance from the center of nature's true state at which the agent believes to observe the state *very* likely.

The following Python function illustrates this observation model.

```
#| export
def plot_observation_function(noise_level = 1,
                             distance = 0.5,
                             x_extend = 3):
    x = np.linspace(-x_extend, x_extend, 1000)
    plt.plot(x, norm.pdf(x, loc=0, scale=noise_level), 'k-')

    x1 = np.linspace(-x_extend, -distance, 1000)
    plt.fill_between(x1, norm.pdf(x1, loc=0, scale=noise_level),
                    color='darkgreen', alpha=0.5)

    x2 = np.linspace(-distance, 0.5, 1000)
    plt.fill_between(x2, norm.pdf(x2, loc=0, scale=noise_level),
                    color='lightgreen', alpha=0.5)

    x3 = np.linspace(0.5, 1+distance, 1000)
    plt.fill_between(x3, norm.pdf(x3, loc=0, scale=noise_level),
                    color='violet', alpha=0.5)

    x4 = np.linspace(1+distance, x_extend, 1000)
    plt.fill_between(x4, norm.pdf(x4, loc=0, scale=noise_level),
                    color='purple', alpha=0.5)

    # Get current xticks and labels
    locations, labels = plt.xticks()

    # Create new labels
    new_labels = ['' for _ in labels]
    for i, loc in enumerate(locations):
        if abs(loc - 0) < 1e-2: new_labels[i] = 'A'
        elif abs(loc - 1) < 1e-2: new_labels[i] = 'B'

    # Set new labels
    plt.xticks(locations, new_labels)
    plt.gca().spines[['top', 'right']].set_visible(False)
    plt.xlim(-x_extend, x_extend)
```

Assuming A is the true state of nature, the shaded areas indicate the probability of observing each signal. The noise level $\sigma = 1.0$ and $d = 0.5$.

```
plot_observation_function(noise_level=1, distance=0.5)
```

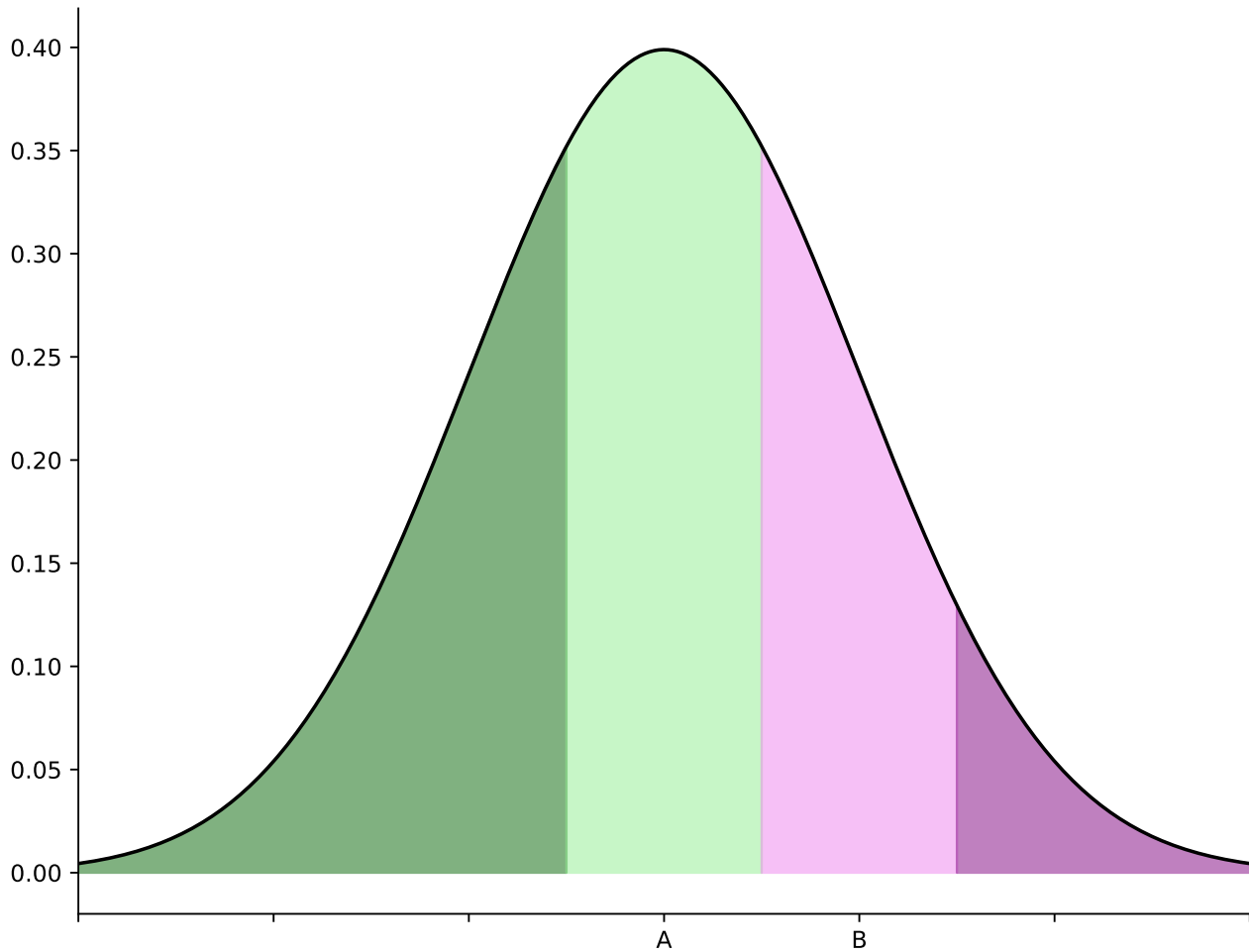


Figure 2.1: Illustration of the observational uncertainty function

We compute these observation probabilities as follows.

```
#|export
def probabilities(noiselevel, likelydistance):
    """
    Returns observation probabilities.
    """
    pVeryLikelyTrue = norm.cdf(-likelydistance, 0, noiselevel)      # A
    pLikelyTrue = norm.cdf(0.5, 0, noiselevel) - pVeryLikelyTrue    # a
    pLikelyWrong = norm.cdf(1+likelydistance, 0, noiselevel)\
        - norm.cdf(0.5, 0, noiselevel)                             # b
    pVeryLikelyWrong = 1 - norm.cdf(1+likelydistance, 0, noiselevel) # B

    return [pVeryLikelyTrue, pLikelyTrue, pLikelyWrong, pVeryLikelyWrong]
```

```
probabilities(noiselevel=2.7, likelydistance=0.5)
```

```
[0.42654189431596573,
```

```
0.1469162113680686,  
0.13728453356199366,  
0.289257360753972]
```

We observe how the observation probabilities change as we change the noise level, σ and the distance d .

```
#|export  
def plot_observation_probabilities(max_noiselevel, likelydistance=0.5):  
    noiselevels = np.linspace(0.01, max_noiselevel, 200)  
    probs = np.array([probabilities(sig, likelydistance) for sig in noiselevels])  
    plt.plot(noiselevels, probs[:,0], '.-', label= 'A', color = 'darkgreen')  
    plt.plot(noiselevels, probs[:,1], '.-', label= 'a', color = 'lightgreen')  
    plt.plot(noiselevels, probs[:,2], '.-', label= 'b', color = 'violet')  
    plt.plot(noiselevels, probs[:,3], '.-', label= 'B', color = 'purple')  
  
    plt.plot(noiselevels, probs[:,0]+probs[:,1], '--', label= 'A+a', color =  
↪ 'green')  
    plt.plot(noiselevels, probs[:,2]+probs[:,3], '--', label= 'B+b', color = 'pink')  
  
    plt.xlabel('Observational uncertainty  $\sigma$ ')  
    plt.ylabel('Observation probabilities')  
    plt.legend()
```

```
plot_observation_probabilities(max_noiselevel=4, likelydistance=0.5)
```

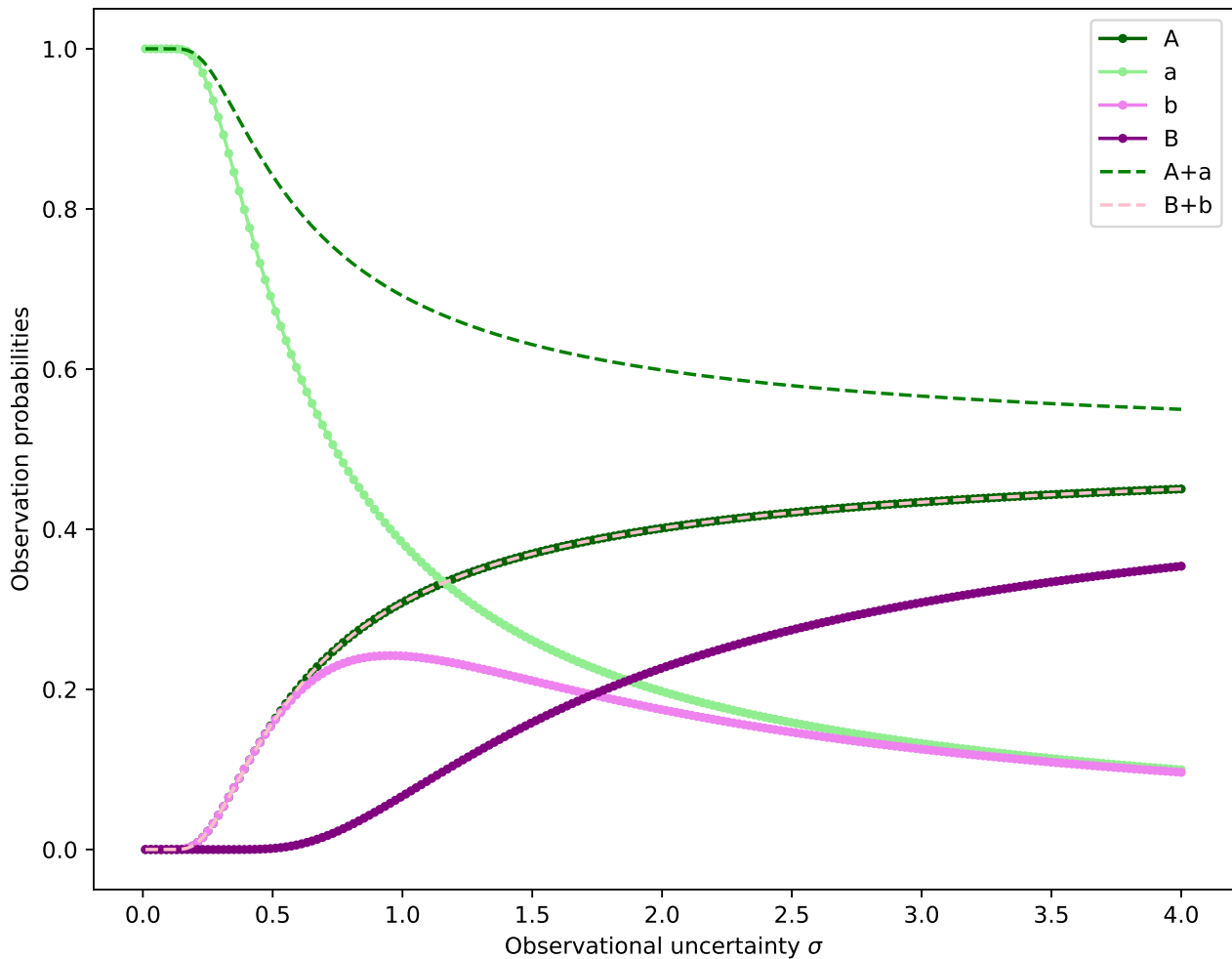



Figure 2.2: Observation probabilities as a function of observational uncertainty

2.2 Environment classes

We first define an environmental base class defining some common attributes and implementing checks for internal consistency.

```
#| export
class ebase(object):
    """Base environment. All environments should inherit from this one."""

    def __init__(self):

        self.T = self.TransitionTensor()
        self.R = self.RewardTensor()
        self.O = self.ObservationTensor()

        self.Aset = self.actions()
        self.Sset = self.states()
        self.Oset = self.observations()

        # CHECKS
```

```

R, T, O = self.R, self.T, self.O

# number of agents
N = R.shape[0]
assert O.shape[0] == N, "Inconsistent number of agents"
assert len(T.shape[1:-1]) == N, "Inconsistent number of agents"
assert len(R.shape[2:-1]) == N, "Inconsistent number of agents"

# number of actions for each agent
M = T.shape[1]
assert np.allclose(T.shape[1:-1], M), 'Inconsistent number of actions'
assert np.allclose(R.shape[2:-1], M), 'Inconsistent number of actions'
assert np.all(list(map(len, self.Aset)) == np.array(M).repeat(N)), \
    'Inconsistent number of actions'

# number of states
Z = T.shape[0]
assert T.shape[-1] == Z, 'Inconsistent number of states'
assert R.shape[-1] == Z, 'Inconsistent number of states'
assert R.shape[1] == Z, 'Inconsistent number of states'
assert O.shape[1] == Z, 'Inconsistent number of states'
assert len(self.Sset) == Z, 'Inconsistent number of states'

# number of observations
Q = O.shape[-1]
assert np.all(list(map(len, self.Oset)) == np.array(Q).repeat(N)), \
    'Inconsistent number of observations'

assert np.allclose(T.sum(-1), 1), 'Transition model wrong'
assert np.allclose(O.sum(-1), 1), 'Observation model wrong'

```

2.3 Single-agent environment

Then, we summarize the observaton logic from above into a single-agent environment Python class `SingleAgentUncertainDecisionProblem` which can be used with the learning dynamics (as implement in the Chapter 3).

```

#| export
class SingleAgentUncertainDecisionProblem(ebase):

    def __init__(self,
                  noiselevel: float, # standard deviation of natural uncertainty
                  probabilityA: float = 0.5, # probability of nature to be in A
                  likelydistance: float = 1): # distance when agent's believe become
↪    *very* likely
        self.env_A = probabilityA
        self.noise = noiselevel
        self.likelydistance = likelydistance
        self.probs = probabilities(noiselevel, likelydistance)

        self.N = 1

```

```

self.M = len(self.actions()[0])
self.Z = len(self.states())
self.Q = len(self.observations()[0])

# initial states
super().__init__()
# self.state = np.random.choice(np.where(self.F==0)[0])

```

In the single-agent environment, the environmental states consist only of nature states.

```

#| export
@patch
def states(self:SingleAgentUncertainDecisionProblem):
    return ["A", "B"]

```

The agent's action set,

```

#| export
@patch
def actions(self:SingleAgentUncertainDecisionProblem):
    return [['A', 'B']]

```

and its observation set,

```

#| export
@patch
def observations(self:SingleAgentUncertainDecisionProblem):
    return [['A', 'a', 'b', 'B']]

```

The transition tensor is very simple. It only depends on the base probability p_A .

```

#| export
@patch
def TransitionTensor(self:SingleAgentUncertainDecisionProblem):
    dim = np.concatenate((
        [self.Z],
        [self.M for _ in range(self.N)],
        [self.Z]))
    Tsas = np.ones(dim) * (-1)

    for index, _ in np.ndenumerate(Tsas):
        Tsas[index] = self._transition_probability(index[0],
                                                    index[1:-1],
                                                    index[-1])

    return Tsas

@patch
def _transition_probability(self:SingleAgentUncertainDecisionProblem,
                           s, jA, s_):
    env_ = self.states()[s_]
    if env_ == 'A':

```

```

        return self.env_A
    else:
        return 1-self.env_A

```

The reward tensor yields,

```

#| export
@patch
def RewardTensor(self:SingleAgentUncertainDecisionProblem):
    """Get the Reward Tensor R[i,s,a1,...,aN,s']."""
    dim = np.concatenate(([self.N],
                           [self.Z],
                           [self.M for _ in range(self.N)],
                           [self.Z]))
    Risas = np.zeros(dim)

    for index, _ in np.ndenumerate(Risas):
        Risas[index] = self._reward(index[0], index[1], index[2:-1], index[-1])
    return Risas

@patch
def _reward(self:SingleAgentUncertainDecisionProblem, i, s, jA, s_):
    # translating states and actions into coordinates
    env = self.states()[s]
    act = self.actions()[i][jA[0]]

    reward = 0.0
    if env == act: reward = 1.0
    elif env != act: reward = -1.0
    return reward

```

The observation tensor is implemented according to the observation model described above,

```

#| export
@patch
def ObservationTensor(self:SingleAgentUncertainDecisionProblem):
    """Get the Observation Tensor O[i,s,o]."""
    Oiso = np.ones((self.N, self.Z, self.Q))

    for index, _ in np.ndenumerate(Oiso):
        Oiso[index] = self._observation(index[0], index[1], index[2])
    return Oiso

@patch
def _observation(self:SingleAgentUncertainDecisionProblem, i, s, o):
    # translating states and actions into coordinates
    env = self.states()[s]
    eobs = self.observations()[i][o]

    # observations probability
    p = 0.0

```

```

    if eobs.upper() == env and eobs.isupper():
        p = self.probs[0]
    if eobs.upper() == env and not eobs.isupper():
        p = self.probs[1]
    if eobs.upper() != env and not eobs.isupper():
        p = self.probs[2]
    if eobs.upper() != env and eobs.isupper():
        p = self.probs[3]

    return p

```

Last, the environment gets an id method return a string identifier of the environment to be used in saving and retrieving results data.

```

#| export
@patch
def id(self:SingleAgentUncertainDecisionProblem):
    """
    Returns id string of environment
    """
    id = f"{self.__class__.__name__}_"+\
        f"{self.noise}_{self.env_A}_{self.likelydistance}"
    return id

```

We execute some basic tests of the environment.

```

env = SingleAgentUncertainDecisionProblem(noiselevel=1.0,
                                          probabilityA=0.5,
                                          likelydistance=0.5)

print(env.id())
print()
print("State set:", env.Sset)
print()
print("Observation set:", env.Oset[0])
print()
print("Action set:", env.Aset[0])
print()
print("Transition tensor:\n", env.TransitionTensor())
print()
print("Observation tensor:\n", env.ObservationTensor())

```

SingleAgentUncertainDecisionProblem_1.0_0.5_0.5

State set: ['A', 'B']

Observation set: ['A', 'a', 'b', 'B']

Action set: ['A', 'B']

Transition tensor:

```
[[[0.5 0.5]
  [0.5 0.5]]
```

```
[[0.5 0.5]
 [0.5 0.5]]]
```

Observation tensor:

```
[[[0.30853754 0.38292492 0.24173034 0.0668072 ]
  [0.0668072 0.24173034 0.38292492 0.30853754]]]
```

2.4 Two-agent environment

In addition to the noisy observations of the nature state, agents in the two-agent environment can also observe the actions of the other agent. Thus, the environment here is composed of the natural environment and the social environment.

In each episode, agents choose their action subsequently. The first agent chooses an action only with the help of the noisy signal of the natural environment. Then, the second agent chooses an action with the help of the noisy signal of the natural environment and the observation of the first agent's action. In each episode the order of agents is randomly chosen.

```
#| export
class TwoAgentUncertainDecisionProblem(ebase):

    def __init__(self,
                  noiselevel: float, # standard deviation of natural uncertainty
                  probabilityA: float = 0.5, # probability of nature to be in A
                  likelydistance: float = 1): # distance when agent's believe become
↪ *very* likely
        assert probabilityA >= 0 and probabilityA <= 1

        self.env_A = probabilityA
        self.noise = noiselevel
        self.likelydistance = likelydistance
        self.probs = probabilities(noiselevel, likelydistance)

        self.N = 2
        self.M = len(self.actions()[0])
        self.Z = len(self.states())
        self.Q = len(self.observations()[0])

        # initial states
        super().__init__()
```

The environmental states are composed of the natural states and the social states. The natural states are identical to the single-agent environment. The social states indicate the position of each agent and the action of the other agent,

```
'12.', '.1A', '.1B', '21.', '.2A', '.2B'
```

For example, the social state 12. indicates that the first agent is behind the second agents which has not chosen any action yet. The social state .1A indicates that the second agent has chosen action *A*, and now, the first agent is about to choose an action relevant to obtain reward or punishment from the environment. In the social state .1B, the second agent has chosen action *B*. When it is not an agents turn to choose, their action simply has no effect and the mechanics of the environment.

```
#| export
@patch
def states(self:TwoAgentUncertainDecisionProblem):
    nAorB = ["A", "B"] # nature's state
    flow = ['12.', '.1A', '.1B', '21.', '.2A', '.2B'] # social state
    return [f'{n}{f}' for n, f in it.product(nAorB, flow)]
```

The agents' action set is identical to the single-agent environment.

```
#| export
@patch
def actions(self:TwoAgentUncertainDecisionProblem):
    AorB = ['A', 'B']
    return [AorB, AorB]
```

The observation set in the two-agent environment is slightly more complicated. Essentially it is composed of the noisy signal of the natural environment (in the first position) and the observation of the other agent's action (in the second position). Additionally, there is a dummy observation, .., for the agent which is currently not active.

```
#| export
@patch
def observations(self:TwoAgentUncertainDecisionProblem):
    obs = ['A.', 'a.', 'b.', 'B.', # for the first agent in line
           'AA', 'aA', 'bA', 'BA', # for the second agent in line
           'AB', 'aB', 'bB', 'BB',
           '..'] # for the inactive agent
    return [obs, obs]
```

The transition tensor is slightly more complicated than in the single-agent environment. It models the movement and signaling of other agent's choices.

```
#| export
@patch
def TransitionTensor(self:TwoAgentUncertainDecisionProblem):
    dim = np.concatenate((self.Z, [self.M for _ in range(self.N)], self.Z))
    Tsas = np.ones(dim) * (-1)

    for index, _ in np.ndenumerate(Tsas):
        Tsas[index] = self._transition(index[0], index[1:-1], index[-1])
    return Tsas

@patch
def _transition(self:TwoAgentUncertainDecisionProblem, s, jA, s_):
    # translating states and actions into coordinates
```

```

nat, pos2, pos1, choice = self.states()[s]          # current state
nat_, pos2_, pos1_, choice_ = self.states()[s_]      # next state

# in movement state:
#   pos2= agent that plays second, pos1=agent that plays, choise='.';

# in final state:
#   pos2='.', pos1=agent that plays, choice= choice of previous agent

i = int(pos1)-1 # acting agent
act = self.actions()[i][jA[i]] # action `A` or `B` of acting agent

prop = 0.0 # default probability

if choice != '.' and choice_ == '.': # "final" states
    if nat_ == 'A':
        prop = self.env_A/2
        # *1/2 because there are 2 possible next states with nat_='A'
    else:
        prop = (1-self.env_A)/2

elif nat == nat_ and pos2 == pos1_: # movement
    if act == choice_: prop = 1.0

return prop

```

The reward tensor is very similar than in the single-agent environment. The only difference is that we scale the reward by a factor of 2 because also the episode length is doubled. In each episode, each agent has to wait for the other agent to make a choice. To have both environment with comparable average rewards, we scale the reward by a factor of 2.

```

#| export
@patch
def RewardTensor(self:TwoAgentUncertainDecisionProblem):
    """Get the Reward Tensor R[i,s,a1,...,aN,s']."""
    dim = np.concatenate(([self.N], [self.Z], [self.M for _ in range(self.N)],
                               [self.Z]))
    Risas = np.zeros(dim)

    for index, _ in np.ndenumerate(Risas):
        Risas[index] = self._reward(index[0], index[1], index[2:-1], index[-1])
    return Risas

@patch
def _reward(self:TwoAgentUncertainDecisionProblem, i, s, jA, s_):
    # translating states and actions into coordinates
    nat, pos2, pos1, choice = self.states()[s]
    nat_, pos2_, pos1_, choice_ = self.states()[s_]

    j = int(pos1) - 1 #acting agent
    act = self.actions()[j][jA[j]] # action `A` or `B` of acting agent

```



```

reward = 0 # default reward

if j == i: # if i is acting agent
    # if act is correct (according to nature)
    if act == nat: reward = 2
    elif act != nat: reward = -2
return reward

```

The observation tensor in the two-agent environment is more complicated than in the single-agent environment, as it combines the noisy observation from the nature state with the observation of the other agent's action.

```

#| export
@patch
def ObservationTensor(self:TwoAgentUncertainDecisionProblem):
    """Returns Oiso. Probability the agent i makes observation o in state s"""
    Oiso = np.ones((self.N, self.Z, self.Q))

    for index, _ in np.ndenumerate(Oiso):
        Oiso[index] = self._observation(index[0], index[1], index[2])
    return Oiso

@patch
def _observation(self:TwoAgentUncertainDecisionProblem, i, s, o):
    # translating states and actions into coordinates
    nat, _, pos1, choice = self.states()[s]
    nobs, sobs = self.observations()[i][o] #nature obs., social obs.

    # observations probability composed of action obs and env obs
    pA, pE = 0.0, 0.0

    if nobs != ".":
        if choice == sobs: #agent observes the true choice of predecessor
            pA = 1.0

        if int(pos1)-1 == i: # only pos1 agent observes something
            if nobs.upper() == nat and nobs.isupper():
                pE = self.probs[0] #prop(A) if env in A, prop(B) if env in B
            if nobs.upper() == nat and not nobs.isupper():
                pE = self.probs[1] #prop(a) if env in A, prop(b) if env in B
            if nobs.upper() != nat and not nobs.isupper():
                pE = self.probs[2] #prop(b) if env in A, prop(a) if env in B
            if nobs.upper() != nat and nobs.isupper():
                pE = self.probs[3] #prop(B) if env in A, prop(A) if env in B

    elif nobs == "." and sobs == ".":
        #agent i is not acting agent -> makes no observation
        if int(pos1)-1 != i: pA = pE = 1.0

    return pE * pA

```

As above, the environment also gets an `id` method return a string identifier of the environment to be

used in saving and retrieving results data.

```
#| export
@patch
def id(self:TwoAgentUncertainDecisionProblem):
    """
    Returns id string of environment
    """
    id = f"{self.__class__.__name__}_"+\
        f"{self.noise}_{self.env_A}_{self.likelydistance}"
    return id
```

Executing some basic tests of the environment,

```
env = TwoAgentUncertainDecisionProblem(noiselevel=1.0,
                                       probabilityA=0.6,
                                       likelydistance=0.5)

print(env.id())
print()
print("State set:", env.Sset)
print()
print("Observation set:", env.Oset[0])
print()
print("Action set:", env.Aset[0])
print()
print("Transition tensor's shape:\n", env.TransitionTensor().shape)
print()
print("Observation tensor's shape:\n", env.ObservationTensor().shape)
```

TwoAgentUncertainDecisionProblem_1.0_0.6_0.5

State set: ['A12.', 'A.1A', 'A.1B', 'A21.', 'A.2A', 'A.2B', 'B12.', 'B.1A', 'B.1B',
↪ 'B21.', 'B.2A', 'B.2B']

Observation set: ['A.', 'a.', 'b.', 'B.', 'AA', 'aA', 'bA', 'BA', 'AB', 'aB', 'bB',
↪ 'BB', '...']

Action set: ['A', 'B']

Transition tensor's shape:
(12, 2, 2, 12)

Observation tensor's shape:
(2, 12, 13)

Finally, we export the environment classes into their own module,

```
import nbdev
nbdev.export.nb_export("j01_ENVI_UncertainDecisionProblem.ipynb", "_code")
```

3 Learning Dynamics

Here, we describe and implement the learning dynamics of our agents. We start by importing the necessary packages and define some configuration for exporting the this notebook to a python module,

```
#| default_exp LearningDynamics
```

```
#| export
import jax
import numpy as np
import jax.numpy as jnp
from jax import jit
from functools import partial
import itertools as it
```

We also define a small helper function to convert a single number into an one-dimensional array, which is useful for some of the agent implementations later on.

```
#| export
def make_variable_vector(variable, # can be iterable or float or int
                        length:int # length of the vector
                        ): # vector
    "Turn a `variable` into a vector or check that `length` is consistent."
    if hasattr(variable, '__iter__'):
        assert len(variable) == length, 'Wrong number given'
        return jnp.array(variable)
    else:
        return jnp.repeat(variable, length)
```

3.1 Model-based learning dynamics

We start with the model-based learning dynamics, which we implement as a Python class. The two main parameters each learning dynamics object receives are an environment object `env` (see Chapter 2) and the learning rates `alpha` of the agents. The remaining parameters have no effect and exist only that the class below can be used as a drop-in replacement for the more complex learning dynamics used in the first author's Python package [pyCRLD](#). The reminder of the `__init__` method ensures that that the environmental dynamics, rewards, and observation are internally coherent.

```
#| export
class LearningAgents:

    def __init__(self,
                 env,
```

```

        learning_rates,
        discount_factors=None, # dummy argument for compatibility
        choice_intensities=None, # dummy argument for compatibility
        use_prefactor=None # dummy argument for compatibility
    ):

self.env = env
TransitionTensor = env.T
RewardTensor = env.R
ObservationTensor = env.O

R = jnp.array(RewardTensor)
T = jnp.array(TransitionTensor)
O = jnp.array(ObservationTensor)

# number of agents
NrAgt = R.shape[0]
assert len(T.shape[1:-1]) == NrAgt, "Inconsistent number of agents"
assert len(R.shape[2:-1]) == NrAgt, "Inconsistent number of agents"
assert O.shape[0] == NrAgt, "Inconsistent number of agents"

# number of actions for each agent
NrAct = T.shape[1]
assert np.allclose(T.shape[1:-1], NrAct), 'Inconsistent number of actions'
assert np.allclose(R.shape[2:-1], NrAct), 'Inconsistent number of actions'

# number of states
NrSts = T.shape[0]
assert T.shape[-1] == NrSts, 'Inconsistent number of states'
assert R.shape[-1] == NrSts, 'Inconsistent number of states'
assert R.shape[1] == NrSts, 'Inconsistent number of states'
assert O.shape[1] == NrSts, 'Inconsistent number of states'

# number of observations
NrObs = O.shape[-1]

self.R, self.T, self.O = R, T, O
self.NrAgt, self.NrAct, self.NrSts, self.NrObs = NrAgt, NrAct, NrSts, NrObs
self.N, self.M, self.Z, self.Q = NrAgt, NrAct, NrSts, NrObs

self.learning_rates = make_variable_vector(learning_rates, NrAgt)

# state and obs distribution helpers
self.Omega = self._OtherAgentsActionsSummationTensor()
self.has_last_statdist = False
self._last_statdist = jnp.ones(NrSts) / NrSts
self.has_last_obsdist = False
self._last_obsdist = jnp.ones((NrAgt, NrObs)) / NrObs

```

We add a method to the class that returns a random strategy for each agent,

```

#| export
def random_softmax_policy(self:LearningAgents):
    """Softmax policy with random probabilities."""
    expQ = jnp.exp(np.random.randn(self.NrAgt, self.NrObs, self.NrAct))
    return expQ / expQ.sum(axis=-1, keepdims=True)

LearningAgents.random_softmax_policy = random_softmax_policy

```

and one, where each agent selects each action with equal probability,

```

#| export
def zero_intelligence_policy(self:LearningAgents):
    """Policy with equal probabilities."""
    return jnp.ones((self.NrAgt, self.NrObs, self.NrAct)) / float(self.NrAct)
LearningAgents.zero_intelligence_policy = zero_intelligence_policy

```

Learning update

The `update` method implements the learning update of the agents' strategy,

$$X_{t+1}^{i,o^i,a^i} = \frac{X_t^{i,o^i,a^i} \exp(\alpha^i R_{X_t}^{i,o^i,a^i})}{\sum_{b \in \mathcal{A}^i} X_t^{i,o^i,b} \exp(\alpha^i R_{X_t}^{i,o^i,b})},$$

```

#| export
@partial(jit, static_argnums=0)
def update(self: LearningAgents,
           Xioa # Joint strategy
           ) -> tuple: # (Updated joint strategy, Prediction error)
    """
    Performs a learning update of the joint behavioral rule,
    given joint behavior `Xioa`.
    """
    Rioa = self.Rioa(Xioa)
    n = jnp.newaxis
    XexpaRioa = Xioa * jnp.exp(self.learning_rates[:,n,n] * Rioa)
    Xioa_ = XexpaRioa / XexpaRioa.sum(-1, keepdims=True)
    return Xioa_

LearningAgents.update = update

```

Rioa | Average rewards

The `Rioa` method computes the average rewards R_X^{i,o^i,a^i} for each agent i , observation o^i , and action a^i . Whenever agent i observes observation o^i , the environment is state s with probability $B^{i,o^i,s}$. In s , all other agents $j \neq i$ behave according to Y_X^{j,s,a^j} . The environment transitions to state $\acute{s}$ with probability

$T^{s,a,\hat{s}}$. And agent i receives reward $R^{i,s,a,\hat{s}}$. Thus, the behavior-average observation-action reward for action a^i under observation o^i reads,

$$R_X^{i,o^i,a^i} = \sum_{s \in \mathcal{S}} \sum_{a^j \in \mathcal{A}^j} \sum_{\hat{s} \in \mathcal{S}} \sum_{j \neq i} B_X^{i,o^i,s} Y_X^{j,s,a^j} T^{s,a,\hat{s}} R^{i,s,a,\hat{s}}.$$

```
#| export
@partial(jit, static_argnums=0)
def Riosa(self: LearningAgents,
          Xioa,
          Bios=None,
          Yisa=None):
    """Compute average reward Riosa, given joint policy X """
    # For speed up
    Bios = self.Bios(Xioa) if Bios is None else Bios
    Yisa = self.Yisa(Xioa) if Yisa is None else Yisa

    # Variables
    # agent i, act a, state s, next state s_, observation o
    i = 0; a = 1; s = 2; s_ = 3; o = 4
    j2k = list(range(5, 5+self.NrAgt-1)) # other agents
    b2d = list(range(5+self.NrAgt-1, 5+self.NrAgt-1 + self.NrAgt)) # all actions
    e2f = list(range(4+2*self.NrAgt, 4+2*self.NrAgt + self.NrAgt-1)) # all other
    ↪ acts

    sumsis = [[j2k[l], s, e2f[l]] for l in range(self.NrAgt-1)] # sum inds
    otherY = list(it.chain(*zip((self.NrAgt-1)*[Yisa], sumsis)))
    args = [self.Omega, [i]+j2k+[a]+b2d+e2f, Bios, [i, o, s]] +\
           otherY + [self.T, [s]+b2d+[s_], self.R, [i, s]+b2d+[s_],
                    [i, o, a]]

    return jnp.einsum(*args, optimize=True)

LearningAgents.Riosa = Riosa
```

Yisa | State strategy

The **Yisa** method computes the state-action strategy Y_X^{i,s,a^i} given the current observation-action policy X^{i,o^i,a^i} as,

$$Y_X^{j,s,a^j} = \sum_{o^j \in \mathcal{O}^j} O^{j,s,o^j} X^{j,o^j,a^j},$$

```
#| export
@partial(jit, static_argnums=0)
def Yisa(self: LearningAgents,
          Xioa):
    """
    Compute state-action policy given the current observation-action policy
    """
    i = 0; a = 1; s = 2; o = 4 # variables
    args = [self.O, [i, s, o], Xioa, [i, o, a], [i, s, a]]
```

```

Yisa = jnp.einsum(*args, optimize=True)

return Yisa

LearningAgents.Yisa = Yisa

```

Bios | State beliefs given observations

The Bios method computes the state beliefs $B_X^{i,o^i,s}$ given the current observation-action policy X^{i,o^i,a^i} as,

$$B_X^{i,o^i,s} = \frac{O^{i,s,o^i} P_X^s}{\sum_s O^{i,s,o^i} P_X^s},$$

```

#| export
@partial(jit, static_argnums=0)
def Bios(self: LearningAgents,
        Xioa):
    """
    Compute 'belief' that environment is in stats s given agent i
    observes observation o (Bayes Rule)

    Unsafe when stationary state distribution is not unique
    (i.e., when policies are too extreme)
    """
    i, s, o = 0, 1, 2 # variables
    # pS = self.statedist(X) # from full obs base (requires Tss from above)
    pS = self.Ps(Xioa, self._last_statedist)
    self._last_statedist = pS
    self.has_last_statdist = True

    b = jnp.einsum(self.O, [i,s,o], pS, [s], [i,s,o], optimize=True)
    bsum = b.sum(axis=1, keepdims=True)
    bsum = bsum + (bsum == 0) # to avoid dividing by zero
    Biso = b / bsum
    Bios = jnp.swapaxes(Biso, 1,-1)

    return Bios

LearningAgents.Bios = Bios

```

Ps | Stationary distribution

The Ps method computes the stationary distribution P_X^s over states given the current joint observation-action strategy X^{i,o^i,a^i} as left eigenvector of the strategy-average transition matrix $T_X^{s,\hat{s}}$,

```

#| export
@partial(jit, static_argnums=0)
def Ps(self,
        Xioa, # Joint strategy
        pS0): # Last stationary state distribution
    """
    Compute stationary state distribution `Ps`, given joint strategy `Xioa`
    using JAX.
    """
    Tss = self.Tss(Xioa)
    _pS = compute_stationarydistribution(Tss)
    nrS = jnp.where(_pS.mean(0) != -10, 1, 0).sum()

    @jit
    def single_dist(pS):
        return jnp.max(jnp.where(_pS.mean(0) != -10,
                                jnp.arange(_pS.shape[0]), -1))

    @jit
    def multi_dist(pS):
        ix = jnp.argmax(jnp.linalg.norm(_pS.T - pS0, axis=-1))
        return ix

    ix = jax.lax.cond(nrS == 1, single_dist, multi_dist, _pS)

    pS = _pS[:, ix]
    return pS

LearningAgents.Ps = Ps

```

Tss | State transition

The Tss method computes the strategy-average transition matrix $T_X^{s,\acute{s}}$ given the current observation-action strategy X^{i,o^i,a^i} as,

$$T_X^{s,\acute{s}} = \sum_{a^j \in \mathcal{A}^j} \prod_{j \in \mathcal{N}} Y_X^{j,s,a^j} T^{s,a,\acute{s}},$$

```

#| export
@partial(jit, static_argnums=0)
def Tss(self: LearningAgents,
        Xioa):
    """Compute average transition model Tss given policy X"""
    Yisa = self.Yisa(Xioa)

    s = 1 # state s
    sprim = 2 # next state s'
    b2d = list(range(3, 3+self.NrAgt)) # all actions

    X4einsum = list(it.chain(*zip(Yisa, [[s, b2d[a]]
                                         for a in range(self.NrAgt)])))
    args = X4einsum + [self.T, [s]+b2d+[sprim], [s, sprim]]

```



```
return jnp.einsum(*args, optimize=True)
```

```
LearningAgents.Tss = Tss
```

We are left with implementing a few helper functions and methods.

aux | Stationary distribution

The `compute_stationarydistribution` function computes the stationary distribution as the left eigenvector of the transition matrix with eigenvalue 1.

```
#!/ export
@jit
def compute_stationarydistribution(Tkk:jnp.ndarray): # Transition matrix
    """Compute stationary distribution for transition matrix `Tkk`."""
    # eigenvectors
    oeival, oeivec = jnp.linalg.eig(Tkk.T)
    oeival = oeival.real
    oeivec = oeivec.real

    get_mask = lambda tol: jnp.abs(oeival - 1) < tol

    tolerances = jax.lax.map(lambda x: 0.1**x, jnp.arange(1,16,1))
    masks = jax.lax.map(get_mask, tolerances)
    ix = jnp.max(jnp.where(masks.sum(-1)>=1, jnp.arange(len(masks)), -1))
    mask = masks[ix]
    tol = tolerances[ix]

    # obtain stationary distribution
    meivec = jnp.where(mask, oeivec, -42)

    dist = meivec / meivec.sum(axis=0, keepdims=True)
    dist = jnp.where(dist < tol, 0, dist)
    dist = dist / dist.sum(axis=0, keepdims=True)

    return jnp.where(meivec== -42, -10, dist)
```

axu | Other agents summation tensor

The `_OtherAgentsActionsSummationTensor` allows to sum over all other agents' actions when computing expected values.

```
#!/ export
def _OtherAgentsActionsSummationTensor(self:LearningAgents):
    """
    To sum over the other agents and their respective actions using `einsum`.
    """
    dim = np.concatenate([self.NrAgt, # agent i
                          [self.NrAgt for _ in range(self.NrAgt-1)], # other agnt
                          [self.NrAct], # agent a of agent i
```

```

        [self.NrAct for _ in range(self.NrAgt)], # all acts
        [self.NrAct for _ in range(self.NrAgt-1)]) # other a's
Omega = np.zeros(dim.astype(int), int)

for index, _ in np.ndenumerate(Omega):
    I = index[0]
    notI = index[1:self.NrAgt]
    A = index[self.NrAgt]
    allA = index[self.NrAgt+1:2*self.NrAgt+1]
    notA = index[2*self.NrAgt+1:]

    if len(np.unique(np.concatenate([I, notI]))) is self.NrAgt:
        # all agents indices are different

        if A == allA[I]:
            # action of agent i equals some other action
            cd = allA[:I] + allA[I+1:] # other actionss
            areequal = [cd[k] == notA[k] for k in range(self.NrAgt-1)]
            if np.all(areequal):
                Omega[index] = 1

    return jnp.array(Omega)
LearningAgents._OtherAgentsActionsSummationTensor =
↪ _OtherAgentsActionsSummationTensor

```

aux | Trajectory

The `trajectory` method simulates a learning trajectory from an initial strategy `Xinit` over a maximum of `Tmax` time steps or until convergence within tolerance `tolerance` is reached.

```

#| export
def trajectory(self: LearningAgents,
               Xinit:jnp.ndarray, # Initial condition
               Tmax:int=100, # the maximum number of iteration steps
               tolerance:float=None, # to determine if a fix point is reached
               verbose=False, # Say something during computation?
               **kwargs) -> tuple: # (`trajectory`, `fixpointreached`)
    """
    Compute a joint learning trajectory.
    """
    traj = []
    t = 0
    X = Xinit.copy()
    fixpreached = False

    while not fixpreached and t < Tmax:
        print(f"\r [computing trajectory] step {t}", end='') if verbose else None
        traj.append(X)

        X_ = self.update(X)
        if np.any(np.isnan(X_)):

```

```

        fixpreached = True
        break

    if tolerance is not None:
        fixpreached = np.linalg.norm(X_ - X) < tolerance

    X = X_
    t += 1

    print(f" [trajectory computed]") if verbose else None

    return np.array(traj), fixpreached
LearningAgents.trajectory = trajectory

```

aux | Average rewards Ri

The Ri method computes the average rewards for each agent given the current strategy,

$$R_X^i = \sum_{o^i \in \mathcal{O}^i} \sum_{a^i \in \mathcal{A}^i} P_X^{o^i} R^{i,o,a}.$$

```

#| export
# @partial(jit, static_argnums=0)
def Ri(self: LearningAgents,
      Xioa):
    """Compute average reward Ri, given joint policy X"""
    i, o, a = 0, 1, 2
    return jnp.einsum(self.obsdist(Xioa), [i, o],
                     Xioa, [i, o, a],
                     self.Rioa(Xioa), [i, o, a], [i])
LearningAgents.Ri = Ri

```

To do so, it uses the observation distribution $P_X^{o^i}$.

aux | Observation distribution

The obsdist method computes the observation distribution for each agent given the current strategy as the left eigenvector of their observation-transitions matrix $T^{i,o^i,\hat{o}^i}$.

```

#| export
# @partial(jit, static_argnums=0)
def obsdist(self: LearningAgents, Xioa):
    """Compute stationary distribution, given joint policy X"""
    Tioo = self.Tioo(Xioa)
    Dio = np.zeros((self.NrAgt, self.NrObs))

    for i in range(self.NrAgt):
        p0 = np.array(compute_stationarydistribution(Tioo[i]))

        p0 = p0[:, p0.mean(0) != -10]

```

```

        if len(p0[0]) == 0: # this happens when the tollerance can distin.
            assert False, 'No _statdist return - must not happen'
        elif len(p0[0]) > 1: # Should not happen, in an ideal world
            # sidenote: This means an ideal world is ergodic ;)
            print("More than 1 state-eigenvector found")
            print(p0.round(2))
            nr = len(p0[0])
            choice = np.random.randint(nr)
            print("taking random one: ", choice)
            p0 = p0[:, choice]

        Dio[i] = p0.flatten()

    self._last_obsdist = Dio
    self.has_last_obsdist = True
    return Dio
LearningAgents.obsdist = obsdist

```

Tioo | Observation transition matrices

The Tioo method computes the observation-transition matrices T^{i,o^i,o^i} for each agent given the current strategy as,

```

#| export
@partial(jit, static_argnums=0)
def Tioo(self, X, Bios=None, Yisa=None):
    """Compute average transition model Tioo, given joint policy X"""
    # For speed up
    Bios = self.Bios(X) if Bios is None else Bios
    Yisa = self.Yisa(X) if Yisa is None else Yisa

    # variables
    # agent i, state s, next state s_, observation o, next obs o', all acts
    i = 0; s = 1; s_ = 2; o = 3; o_ = 4; b2d = list(range(5, 5+self.NrAgt))

    Y4einsum = list(it.chain(*zip(Yisa,
                                   [[s, b2d[a]] for a in range(self.NrAgt)])))

    args = [Bios, [i, o, s]] + Y4einsum + [self.T, [s]+b2d+[s_],
                                                  self.O, [i, s_, o_], [i, o, o_]]
    return jnp.einsum(*args, optimize=True)
LearningAgents.Tioo = Tioo

```

aux | Identifier

The id method returns a string identifier for the learning dynamics object.

```

#| export
def id(self: LearningAgents
    ) -> str: # id

```

```

    """Returns an identifier to handle simulation runs."""
    envid = self.env.id() + "__"
    agentsid = f"j{self.__class__.__name__}_"

    agentsid += 'PartObs_'

    agentsid += f"{str(self.learning_rates)}"

    return envid + agentsid
LearningAgents.id = id

```

3.2 Model-free learning dynamics

Having implemented the model-based learning dynamics, we can now easily implement model-free learning dynamics by inheriting from the model-based class and overriding only the `update` method.

```

#| export
class ModelFreeAgents(LearningAgents):

    # @partial(jit, static_argnums=0)
    def update(self,
                Xioa # Joint strategy
                ) -> tuple: # (Updated joint strategy, Prediction error)
        """
        Performs a learning update of the joint behavioral rule,
        given joint behavior `Xioa`.
        """
        Rioa = self.Rioa(Xioa)
        Pio = self.obsdist(Xioa)

        n = jnp.newaxis
        XexpaRioa = Xioa * jnp.exp(self.learning_rates[:,n,n]
                                   * Pio[:,n,n]
                                   * Xioa
                                   * Rioa)
        Xioa_ = XexpaRioa / XexpaRioa.sum(-1, keepdims=True)
        return Xioa_

    LearningAgents.update = update

```

3.3 Testing the implementation

We test our implementation of the learning dynamics classes on the uncertain decision problem defined in Chapter 2.

```

np.random.seed(42)

```

```

from _code.UncertainDecisionProblem import \
    SingleAgentUncertainDecisionProblem, TwoAgentUncertainDecisionProblem

envparams = dict(probabilityA=0.5, likelydistance=0.5)
envclass = TwoAgentUncertainDecisionProblem
envclassS = SingleAgentUncertainDecisionProblem
env = envclass(noiselevel=2.5, **envparams)
envS = envclassS(noiselevel=2.5, **envparams)

print(env.T.shape)
print(env.R.shape)
print(env.O.shape)

```

```

(12, 2, 2, 12)
(2, 12, 2, 2, 12)
(2, 12, 13)

```

Testing the model-based leaning dynamics

```

MBA_LA = LearningAgents(env, learning_rates=0.25)
print(MBA_LA.id())
MBA_LAS = LearningAgents(envS, learning_rates=0.25)
print(MBA_LAS.id())

```

```

TwoAgentUncertainDecisionProblem_2.5_0.5_0.5__jLearningAgents_PartObs_[0.25 0.25]
SingleAgentUncertainDecisionProblem_2.5_0.5_0.5__jLearningAgents_PartObs_[0.25]

```

```

for LA in [MBA_LA, MBA_LAS]:
    Xioa = LA.random_softmax_policy()
    Xtioa, fpr = LA.trajectory(Xioa, Tmax=5000, tolerance=1e-6)
    print(Xtioa.shape, fpr)

```

```

(306, 2, 13, 2) True
(523, 1, 4, 2) True

```

Testing the model-free leaning dynamics

```

MFA_LA = ModelFreeAgents(env, learning_rates=0.25)
print(MFA_LA.id())
MFA_LAS = ModelFreeAgents(envS, learning_rates=0.25)
print(MFA_LAS.id())

```

```

TwoAgentUncertainDecisionProblem_2.5_0.5_0.5__jModelFreeAgents_PartObs_[0.25 0.25]
SingleAgentUncertainDecisionProblem_2.5_0.5_0.5__jModelFreeAgents_PartObs_[0.25]

```

```
for LA in [MFA_LA, MFA_LAS]:
    Xioa = LA.random_softmax_policy()
    Xtioa, fpr = LA.trajectory(Xioa, Tmax=500000, tolerance=1e-6)
    print(Xtioa.shape, fpr)
```

(10696, 2, 13, 2) True

(5893, 1, 4, 2) True

Finally, we export the learning dynamics classes into their own module,

```
import nbdev
nbdev.export.nb_export("j02_AGEN_LearningDynamics.ipynb", "_code")
```

4 Result | Final rewards

Here, we ask the question, how the level of reward outperformance at convergences depend on the environmental uncertainty and the learning model. To do so, we compare the final rewards obtained by agents in the single-agent and the two-agent uncertain decision problem across different levels of environmental uncertainty and both learning models (model-free vs. model-based). We define the reward outperformance as the difference between the two-agent reward R_{two} , and the single-agent reward R_{single} , normalized by the single-agent reward, $(R_{\text{two}} - R_{\text{single}})/R_{\text{single}}$. We start by implementing the necessary imports and configuration for plotting,

```
import matplotlib.pyplot as plt
import matplotlib.colors as colors
from matplotlib.lines import Line2D

from _code.LearningDynamics import LearningAgents, ModelFreeAgents
from _code.SimulationScripts import create_consistent_linspace
from _code.SimulationScripts import obtain_noise_variation_comparison
from _code.SimulationScripts import collective_intelligence
```

The simulation results are cached in the following folders,

```
#| export
lowlevel_datadictionary = "/Users/wolf/Downloads/L2SocialData"
highlevel_datadictionary = "./_data"
```

We simulate over the following observational noise levels,

```
#| export
noise_levels = create_consistent_linspace(0.01, 4.0, 81)
noise_levels
```

```
array([0.01      , 0.059875, 0.10975 , 0.159625, 0.2095   , 0.259375,
       0.30925 , 0.359125, 0.409   , 0.458875, 0.50875 , 0.558625,
       0.6085  , 0.658375, 0.70825 , 0.758125, 0.808    , 0.857875,
       0.90775 , 0.957625, 1.0075  , 1.057375, 1.10725 , 1.157125,
       1.207   , 1.256875, 1.30675 , 1.356625, 1.4065   , 1.456375,
       1.50625 , 1.556125, 1.606    , 1.655875, 1.70575 , 1.755625,
       1.8055  , 1.855375, 1.90525 , 1.955125, 2.005    , 2.054875,
       2.10475 , 2.154625, 2.2045  , 2.254375, 2.30425 , 2.354125,
       2.404   , 2.453875, 2.50375 , 2.553625, 2.6035   , 2.653375,
       2.70325 , 2.753125, 2.803    , 2.852875, 2.90275 , 2.952625,
       3.0025  , 3.052375, 3.10225 , 3.152125, 3.202    , 3.251875,
       3.30175 , 3.351625, 3.4015  , 3.451375, 3.50125 , 3.551125,
       3.601   , 3.650875, 3.70075 , 3.750625, 3.8005   , 3.850375,
       3.90025 , 3.950125, 4.        ])
```


Obtaining the data

We summarize the cache folders into a parameter dictionary,

```
ddirs = dict(lowlevel_ddir=lowlevel_datadictionary,
             highlevel_ddir=highlevel_datadictionary)
```

Model-based agents

All model-based agents share the following parameters,

```
mobaparams = dict(agentclass=LearningAgents,
                  maxT=5000,
                  number_of_initial_policies_k=25)
```

```
NoiVar_MBA_pA05 = obtain_noise_variation_comparison(
    probabilityA=0.5,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initialX128421
↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
```

```
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initialX779
↪ 4212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA052 = obtain_noise_variation_comparison(
    probabilityA=0.52,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.52_likelydistance0.5_LearningAgents_learning_rates0.25_initialX12842
↪ 151645801287464_maxT5000_Noises7752999609125433264.npz
```

```
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.52_likelydistance0.5_LearningAgents_learning_rates0.25_initialX77
↪ 94212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA055 = obtain_noise_variation_comparison(
    probabilityA=0.55,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.55_likelydistance0.5_LearningAgents_learning_rates0.25_initialX12842
↪ 151645801287464_maxT5000_Noises7752999609125433264.npz
```

```
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.55_likelydistance0.5_LearningAgents_learning_rates0.25_initialX77
↪ 94212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA058 = obtain_noise_variation_comparison(
    probabilityA=0.58,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initalX12842
↳ 151645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initalX77
↳ 94212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA06 = obtain_noise_variation_comparison(
    probabilityA=0.6,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.6_likelydistance0.5_LearningAgents_learning_rates0.25_initalX128421
↳ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.6_likelydistance0.5_LearningAgents_learning_rates0.25_initalX779
↳ 4212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA07 = obtain_noise_variation_comparison(
    probabilityA=0.7,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initalX128421
↳ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initalX779
↳ 4212447693859915_maxT5000_Noises7752999609125433264.npz
```

```
NoiVar_MBA_pA09 = obtain_noise_variation_comparison(
    probabilityA=0.9,
    noise_levels=noise_levels,
    **mobaparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.9_likelydistance0.5_LearningAgents_learning_rates0.25_initalX128421
↳ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.9_likelydistance0.5_LearningAgents_learning_rates0.25_initalX779
↳ 4212447693859915_maxT5000_Noises7752999609125433264.npz
```

Model-free agents

All model-free agents share the following parameters,

```
mofeparams = dict(agentclass=ModelFreeAgents,
                  maxT=250000,
                  number_of_initial_policies_k=10)
```

```
NoiVar_MFA_pA5 = obtain_noise_variation_comparison(
    probabilityA=0.5,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX12631
↪ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX91
↪ 45693657658906017_maxT250000_Noises7752999609125433264.npz
```

```
NoiVar_MFA_pA52 = obtain_noise_variation_comparison(
    probabilityA=0.52,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.52_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX1263
↪ 1382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.52_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX9
↪ 145693657658906017_maxT250000_Noises7752999609125433264.npz
```

```
NoiVar_MFA_pA55 = obtain_noise_variation_comparison(
    probabilityA=0.55,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.55_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX1263
↪ 1382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.55_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initatX9
↪ 145693657658906017_maxT250000_Noises7752999609125433264.npz
```

```
NoiVar_MFA_pA058 = obtain_noise_variation_comparison(
    probabilityA=0.58,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)
```

```

Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX1263
↳ 1382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX9
↳ 145693657658906017_maxT250000_Noises7752999609125433264.npz

```

```

NoiVar_MFA_pA06 = obtain_noise_variation_comparison(
    probabilityA=0.6,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)

```

```

Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.6_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX12631
↳ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.6_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX91
↳ 45693657658906017_maxT250000_Noises7752999609125433264.npz

```

```

NoiVar_MFA_pA07 = obtain_noise_variation_comparison(
    probabilityA=0.7,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)

```

```

Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX12631
↳ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX91
↳ 45693657658906017_maxT250000_Noises7752999609125433264.npz

```

```

NoiVar_MFA_pA09 = obtain_noise_variation_comparison(
    probabilityA=0.9,
    noise_levels=noise_levels,
    **mofeparams, **ddirs)

```

```

Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.9_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX12631
↳ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↳ _probabilityA0.9_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX91
↳ 45693657658906017_maxT250000_Noises7752999609125433264.npz

```

Visualizing the reward outperformance

Finally, we can visualize the reward outperformance for different levels of environmental uncertainty and both learning models.

```

baseProbabilities = [0.5, 0.52, 0.55, 0.58,
                     0.6, 0.7, 0.9]
NoiVar_MBA_list = [NoiVar_MBA_pA05, NoiVar_MBA_pA052, NoiVar_MBA_pA055,
                   ↪ NoiVar_MBA_pA058,
                     NoiVar_MBA_pA06, NoiVar_MBA_pA07, NoiVar_MBA_pA09]
NoiVar_MFA_list = [NoiVar_MFA_pA5, NoiVar_MFA_pA52, NoiVar_MFA_pA55,
                   ↪ NoiVar_MFA_pA058,
                     NoiVar_MFA_pA06, NoiVar_MFA_pA07, NoiVar_MFA_pA09]

fsf = 0.55
fig, ax = plt.subplots(1, 1, figsize=(fsf*16, fsf*6))
cmap = plt.cm.plasma
norm = colors.Normalize(vmin=0.1, vmax=0.5)

ax.set_xlabel('Observational uncertainty  $\sigma$ ')
ax.set_ylabel('Reward outperformance\nat convergence');

for NoiVar_MBA, NoiVar_MFA, baseProbability in zip(NoiVar_MBA_list[:-1],
                                                    NoiVar_MFA_list[:-1],
                                                    baseProbabilities[:-1]):

    col = cmap(norm(1-baseProbability))
    CInk_MBA = collective_intelligence(NoiVar_MBA.ta_Rnki.mean(-1),
                                       NoiVar_MBA.sa_Rnki.mean(-1))
    ax.plot(noise_levels, CInk_MBA.mean(-1), marker="+", ms=7, lw=0.5, color=col,
            label=f'base probability: {baseProbability}')

    CInk_MFA = collective_intelligence(NoiVar_MFA.ta_Rnki.mean(-1),
                                       NoiVar_MFA.sa_Rnki.mean(-1))
    ax.plot(noise_levels, CInk_MFA.mean(-1), marker="x", ms=7, lw=0.5, color=col,
            label=f'base probability: {baseProbability}')

# Add colorbar
sm = plt.cm.ScalarMappable(cmap=cmap, norm=norm)
cbar = plt.colorbar(sm, ax=ax, pad=0.025)
cbar.set_label('State uncertainty  $p_B=1-p_A$ ')
cbar.ax.set_ylim(0.098, 0.502)

for bP in baseProbabilities:
    cbar.ax.plot([0.1, 0.8], [1-bP, 1-bP], color='springgreen', lw=3)

# Add legend to showcase MBA and MFA markers
prms = dict(ls="", color='k', ms=8)
legend_elements = [
    Line2D([0], [0], marker='+', **prms, label='Model-based agents'),
    Line2D([0], [0], marker='x', **prms, label='Model-free agents')]
ax.legend(handles=legend_elements, loc='lower right');

plt.tight_layout()
plt.subplots_adjust(right=1.05)
plt.savefig("./_figs/FinalRewardOutperformance.png", dpi=300)

```

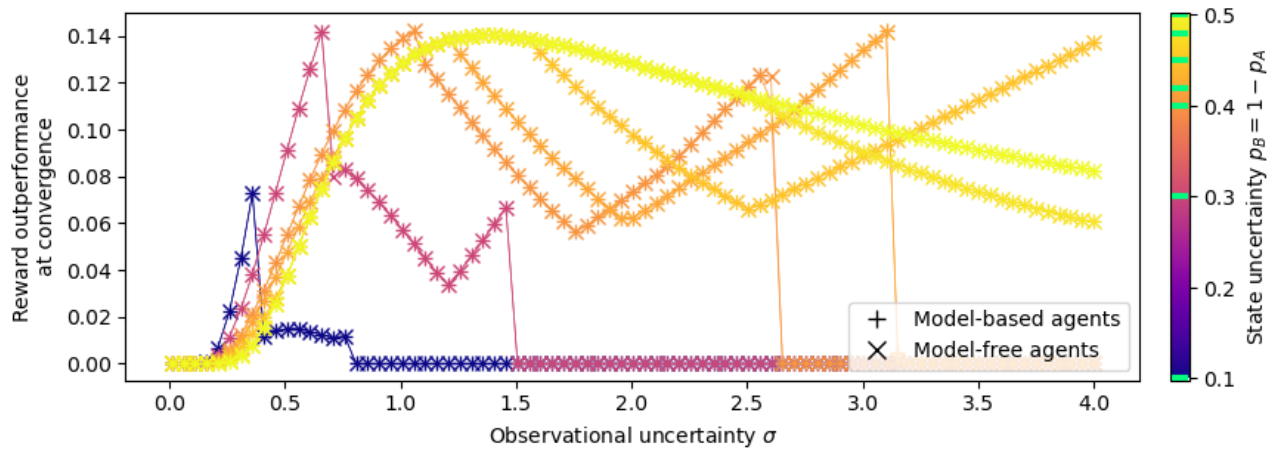


Figure 4.1: Final reward outperformance versus observational uncertainty for different state uncertainties and both learning dynamics

5 Result | Final strategies

Here, we visualize the final strategies across different levels of environmental uncertainty. We start by importing the required packages, defining some configurations and obtaining the required simulation data.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.gridspec import GridSpec
import matplotlib.gridspec as gridspec
from matplotlib.ticker import MultipleLocator
plt.rcParams['text.usetex'] = True

from _code.SimulationScripts import collective_intelligence
from _code.ObtainNoiseVariationResults import
    ↪ obtain_all_noise_variation_comparisons, noise_levels
(NoiVar_MBA_pAhigh, NoiVar_MFA_pAhigh,
 NoiVar_MBA_pAmid, NoiVar_MFA_pAmid,
 NoiVar_MBA_pAlow, NoiVar_MFA_pAlow) = obtain_all_noise_variation_comparisons()
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initialX128421
    ↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initialX779
    ↪ 4212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX12631
    ↪ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX91
    ↪ 45693657658906017_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initialX12842
    ↪ 151645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initialX77
    ↪ 94212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX1263
    ↪ 1382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX9
    ↪ 145693657658906017_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initialX128421
    ↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
```



```

Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem_
↳ _probabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initiaX779
↳ 4212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initiaX12631
↳ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem_
↳ _probabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initiaX91
↳ 45693657658906017_maxT250000_Noises7752999609125433264.npz

```

Final strategies

First, we create a function to visualize the final strategies from a noise-variation dataset `NoiVar`.

```

def plot_final_behaviors(NoiVar, noise_levels, ax_sa=None, ax_ta=None):
    if ax_sa is None and ax_ta is None:
        fsf = 0.7
        fig = plt.figure(figsize=(fsf*6, fsf*5))
        gs = gridspec.GridSpec(2, 1, height_ratios=[1, 3], hspace=0.3)
        ax_sa = fig.add_subplot(gs[0])
        ax_ta = fig.add_subplot(gs[1])

        ax_ta.set_xlabel('Observational uncertainty  $\sigma$ ')

    n = np.newaxis
    # Single agent policies
    sa_finalpolicies_Xon = NoiVar.sa_Xnkioa[..., 0].mean(axis=(1,2)).T
    sa_envsignalaction_o = np.array([1, 1, 0, 0])
    sa_finalpolicies_Xon = np.abs(sa_finalpolicies_Xon - sa_envsignalaction_o[:,n])
    sa_finalpolicies_Xon = sa_finalpolicies_Xon.astype(np.float64)

    sa_ma0set = np.array(['\\texttt{A\\underline{.}}', '\\texttt{a\\underline{.}}',
                          '\\texttt{b\\underline{.}}', '\\texttt{B\\underline{.}}'])
    ax_sa.pcolormesh(noise_levels, np.arange(4), sa_finalpolicies_Xon, cmap='Blues',
↳ vmin=0.0, vmax=1.0)
    ax_sa.invert_yaxis()
    ax_sa.set_yticks(np.arange(0, 4))
    ax_sa.set_yticklabels(sa_ma0set);

    # Two agent policies
    ta_finalpolicies_Xon = NoiVar.ta_Xnkioa[..., 0].mean(axis=(1,2)).T[:12]
    ta_envsignalaction_o = np.array([1, 1, 0, 0, 1, 1, 0, 0, 1, 1, 0, 0])
    ta_finalpolicies_Xon = np.abs(ta_finalpolicies_Xon - ta_envsignalaction_o[:,n])
    ta_finalpolicies_Xon = ta_finalpolicies_Xon.astype(np.float64)

    ta_ma0set = np.array(['\\texttt{A\\underline{.}}', '\\texttt{a\\underline{.}}',
                          '\\texttt{b\\underline{.}}', '\\texttt{B\\underline{.}}',
                          '\\texttt{A\\underline{A}}', '\\texttt{a\\underline{A}}',
                          '\\texttt{b\\underline{A}}', '\\texttt{B\\underline{A}}',
                          '\\texttt{A\\underline{B}}', '\\texttt{a\\underline{B}}',
                          '\\texttt{b\\underline{B}}', '\\texttt{B\\underline{B}}',

```



```

        '..'])
ax_ta.pcolormesh(noise_levels, np.arange(12), ta_finalpolicies_Xon,
↪ cmap='Oranges', vmin=0.0, vmax=1.0)
ax_ta.invert_yaxis()
ax_ta.set_yticks(np.arange(0, 12))
ax_ta.set_yticklabels(ta_ma0set[:12]);
# ax_ta.annotate("Action choice not following nature's signal", xy=(-0.062,
↪ 0.74),
                    # rotation=90, ha='center',va='center', xycoords='axes
↪ fraction')

xlim=[-0.05, max(noise_levels)+0.05]
ax_ta.plot(xlim, [3.5,3.5], '-', color='gray', lw=0.5)
ax_ta.plot(xlim, [7.5,7.5], '-', color='gray', lw=0.5)
ax_ta.plot(xlim, [11.5,11.5], '-', color='gray', lw=0.5)
ax_ta.set_xlim(xlim); ax_sa.set_xlim(xlim);

ax_sa.xaxis.set_minor_locator(MultipleLocator(0.2))
ax_ta.xaxis.set_minor_locator(MultipleLocator(0.2))

```

This function yields, for example,

```
plot_final_behaviors(NoiVar_MBA_pAhigh, noise_levels)
```

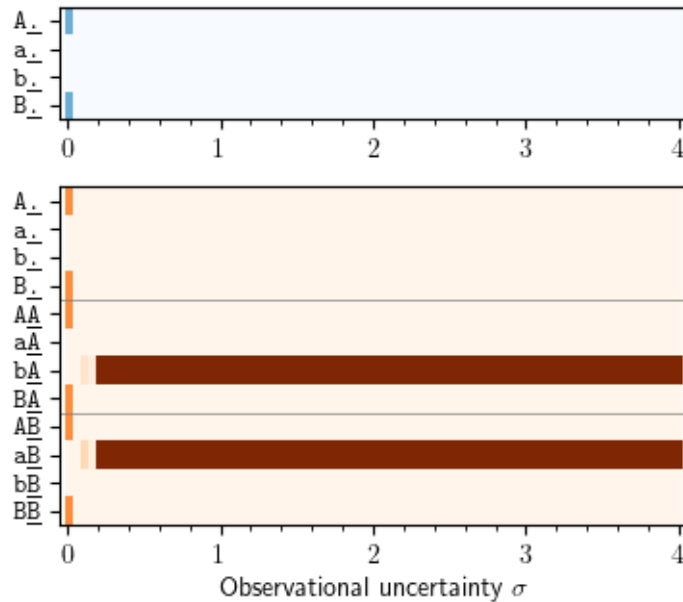


Figure 5.1: Example plot for final strategies versus observational uncertainty.

Reward outperformance

Second, we create a function to visualize the reward outperformance from a noise-variation dataset NoiVar.

```
def plot_reward_outperformance(NoiVar, noise_levels, ax=None):
    if ax is None:
        fsf = 0.7
        fig = plt.figure(figsize=(fsf*6, fsf*4))
        ax = fig.add_subplot(1,1,1)
        ax.set_xlabel('Observational uncertainty  $\sigma$ ')
        ax.set_ylabel('Final reward outperformance');

    CInk = collective_intelligence(NoiVar.ta_Rnki.mean(-1),
                                   NoiVar.sa_Rnki.mean(-1))
    ax.plot(noise_levels, CInk.mean(-1), marker=".", ms=2, lw=0.5, color="k")
    xlim=[-0.05, max(noise_levels)+0.05]
    ax.set_xlim(xlim); ax.set_ylim([-0.005, 0.145])
    ax.xaxis.set_minor_locator(MultipleLocator(0.2))
```

This function yields, for example,

```
plot_reward_outperformance(NoiVar_MBA_pAhigh, noise_levels)
```

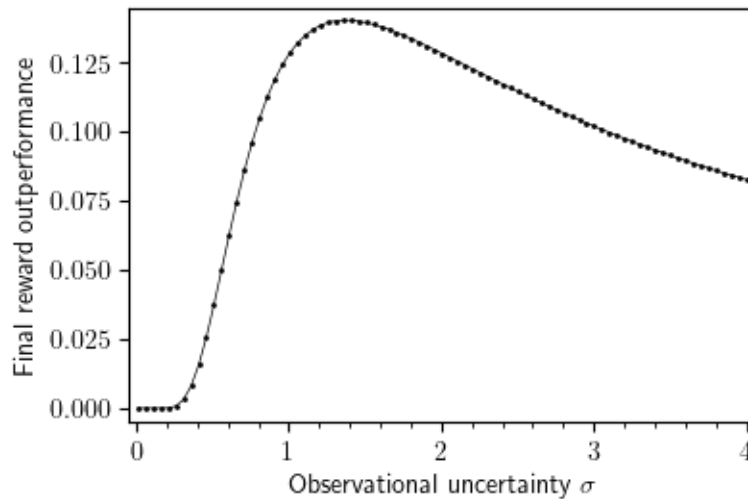


Figure 5.2: Example plot for reward outperformance versus observational uncertainty.

We define two helper functions to plot vertical lines to better visualize the critical levels of observational uncertainty where strategy transitions occur.

```
def plot_vertical_line_at_noise_level(ax, noise_level, ls='-'):
    ax.plot([noise_level, noise_level], ax.get_ylim(), ls, color='gray', lw=0.5)
```

```
def plot_vertical_lines_at_all_axes(axes, noise_level, ls='-'):
    for ax in axes: plot_vertical_line_at_noise_level(ax, noise_level, ls)
```

Putting everything together, we create the Figure 3 of the main text showing the learned strategies.

```

fsf = 0.34
fig = plt.figure(figsize=(fsf*24, fsf*18))

gs = GridSpec(3, 3, hspace=0.15, wspace=0.2,
              height_ratios=[1.2, 4, 4], width_ratios=[1, 1.35, 1])

titleparams = dict(loc="left")
ABCparams = dict(xy=(-0.02, 1.65), ha='right', va='top',
                 xycoords='axes fraction', fontsize=12, fontweight='bold')
agentmodelparams = dict(xy=(-0.26, 0.7), rotation=90, fontsize=14, ha='center',
                        va='center', xycoords='axes fraction', fontweight='bold')
actchoiceparams = dict(xy=(-0.18, 0.7), rotation=90, fontsize=7, ha='center',
                      va='center', xycoords='axes fraction')

## 1 | Highest state uncertainty (p(A)=0.5)
ax_rop_high = fig.add_subplot(gs[0, 0]);
ax_rop_high.set_title("Highest state uncertainty\n $p_B=0.5$ ", **titleparams)
ax_rop_high.annotate("A", **ABCparams)
plot_reward_outperformance(NoiVar_MBA_pAhigh, noise_levels, ax_rop_high)
ax_rop_high.set_xticklabels([]);
ax_rop_high.spines[['top', 'right']].set_visible(False)

gs_mfa_high = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[1,0],
                                                height_ratios=[1, 3], hspace=0.1)
ax_mfa_high_sa = fig.add_subplot(gs_mfa_high[0])
ax_mfa_high_ta = fig.add_subplot(gs_mfa_high[1])
plot_final_behaviors(NoiVar_MFA_pAhigh, noise_levels, ax_mfa_high_sa,
                    ↪ ax_mfa_high_ta)
ax_mfa_high_ta.annotate("Action choice not following nature's signal",
                      **actchoiceparams)
ax_mfa_high_ta.annotate("Model-free agents", **agentmodelparams)
ax_mfa_high_sa.set_xticklabels([]);
ax_mfa_high_ta.set_xticklabels([]);

gs_mba_high = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[2,0],
                                                height_ratios=[1, 3], hspace=0.1)
ax_mba_high_sa = fig.add_subplot(gs_mba_high[0])
ax_mba_high_ta = fig.add_subplot(gs_mba_high[1])
plot_final_behaviors(NoiVar_MBA_pAhigh, noise_levels, ax_mba_high_sa,
                    ↪ ax_mba_high_ta)
ax_mba_high_ta.annotate("Action choice not following nature's signal",
                      **actchoiceparams)
ax_mba_high_ta.annotate("Model-based agents", **agentmodelparams)
ax_mba_high_sa.set_xticklabels([]);
ax_mba_high_ta.set_xlabel("Observational uncertainty  $\sigma$ ");

axes = [ax_rop_high, ax_mfa_high_sa, ax_mfa_high_ta, ax_mba_high_sa, ax_mba_high_ta]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.19)

```

```

## 2 | Medium state uncertainty (p(A)=0.6)
ax_rop_mid = fig.add_subplot(gs[0, 1])
ax_rop_mid.set_title("Medium state uncertainty\n$p_B=0.42$", **titleparams)
ax_rop_mid.annotate("B)", **ABCparams)
plot_reward_outperformance(NoiVar_MBA_pAmid, noise_levels, ax_rop_mid)
ax_rop_mid.set_xticklabels([]);
ax_rop_mid.spines[['top', 'right']].set_visible(False)

gs_mfa_mid = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[1,1],
                                              height_ratios=[1, 3], hspace=0.1)
ax_mfa_mid_sa = fig.add_subplot(gs_mfa_mid[0])
ax_mfa_mid_ta = fig.add_subplot(gs_mfa_mid[1])
plot_final_behaviors(NoiVar_MFA_pAmid, noise_levels, ax_mfa_mid_sa, ax_mfa_mid_ta)
ax_mfa_mid_sa.set_xticklabels([]);
ax_mfa_mid_ta.set_xticklabels([]);

gs_mba_mid = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[2,1],
                                              height_ratios=[1, 3], hspace=0.1)
ax_mba_mid_sa = fig.add_subplot(gs_mba_mid[0])
ax_mba_mid_ta = fig.add_subplot(gs_mba_mid[1])
plot_final_behaviors(NoiVar_MBA_pAmid, noise_levels, ax_mba_mid_sa, ax_mba_mid_ta)
ax_mba_mid_sa.set_xticklabels([]);
ax_mba_mid_ta.set_xlabel("Observational uncertainty  $\sigma$ ");

axes = [ax_rop_mid, ax_mfa_mid_sa, ax_mfa_mid_ta, ax_mba_mid_sa, ax_mba_mid_ta]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.78)
plot_vertical_lines_at_all_axes(axes, 1.22)
plot_vertical_lines_at_all_axes(axes, 1.98)
plot_vertical_lines_at_all_axes(axes, 2.68)
plot_vertical_lines_at_all_axes(axes, 3.11)
plot_vertical_lines_at_all_axes(axes, 3.38, ls="--")

## 3 | Lowest state uncertainty (p(A)=0.7)
ax_rop_low = fig.add_subplot(gs[0, 2])
ax_rop_low.set_title("Low state uncertainty\n$p_B=0.3$", **titleparams)
ax_rop_low.annotate("C)", **ABCparams)
plot_reward_outperformance(NoiVar_MBA_pAlow, noise_levels, ax_rop_low)
ax_rop_low.set_xticklabels([]);
ax_rop_low.spines[['top', 'right']].set_visible(False)

gs_mfa_low = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[1,2],
                                              height_ratios=[1, 3], hspace=0.1)
ax_mfa_low_sa = fig.add_subplot(gs_mfa_low[0])
ax_mfa_low_ta = fig.add_subplot(gs_mfa_low[1])
plot_final_behaviors(NoiVar_MFA_pAlow, noise_levels, ax_mfa_low_sa, ax_mfa_low_ta)
ax_mfa_low_sa.set_xticklabels([]);
ax_mfa_low_ta.set_xticklabels([]);

```

```

gs_mba_low = gridspec.GridSpecFromSubplotSpec(2, 1, subplot_spec=gs[2,2],
                                              height_ratios=[1, 3], hspace=0.1)
ax_mba_low_sa = fig.add_subplot(gs_mba_low[0])
ax_mba_low_ta = fig.add_subplot(gs_mba_low[1])
plot_final_behaviors(NoiVar_MBA_pAllow, noise_levels, ax_mba_low_sa, ax_mba_low_ta)
ax_mba_low_sa.set_xticklabels([]);
ax_mba_low_ta.set_xlabel("Observational uncertainty  $\sigma$ ");

axes = [ax_rop_low, ax_mfa_low_sa, ax_mfa_low_ta, ax_mba_low_sa, ax_mba_low_ta]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.68)
plot_vertical_lines_at_all_axes(axes, 1.18)
plot_vertical_lines_at_all_axes(axes, 1.23)
plot_vertical_lines_at_all_axes(axes, 1.48)
plot_vertical_lines_at_all_axes(axes, 1.63, ls="--")
plot_vertical_lines_at_all_axes(axes, 2.13, ls="--")

plt.savefig("./_figs/FinalBehaviors.png", bbox_inches='tight', dpi=300)

```

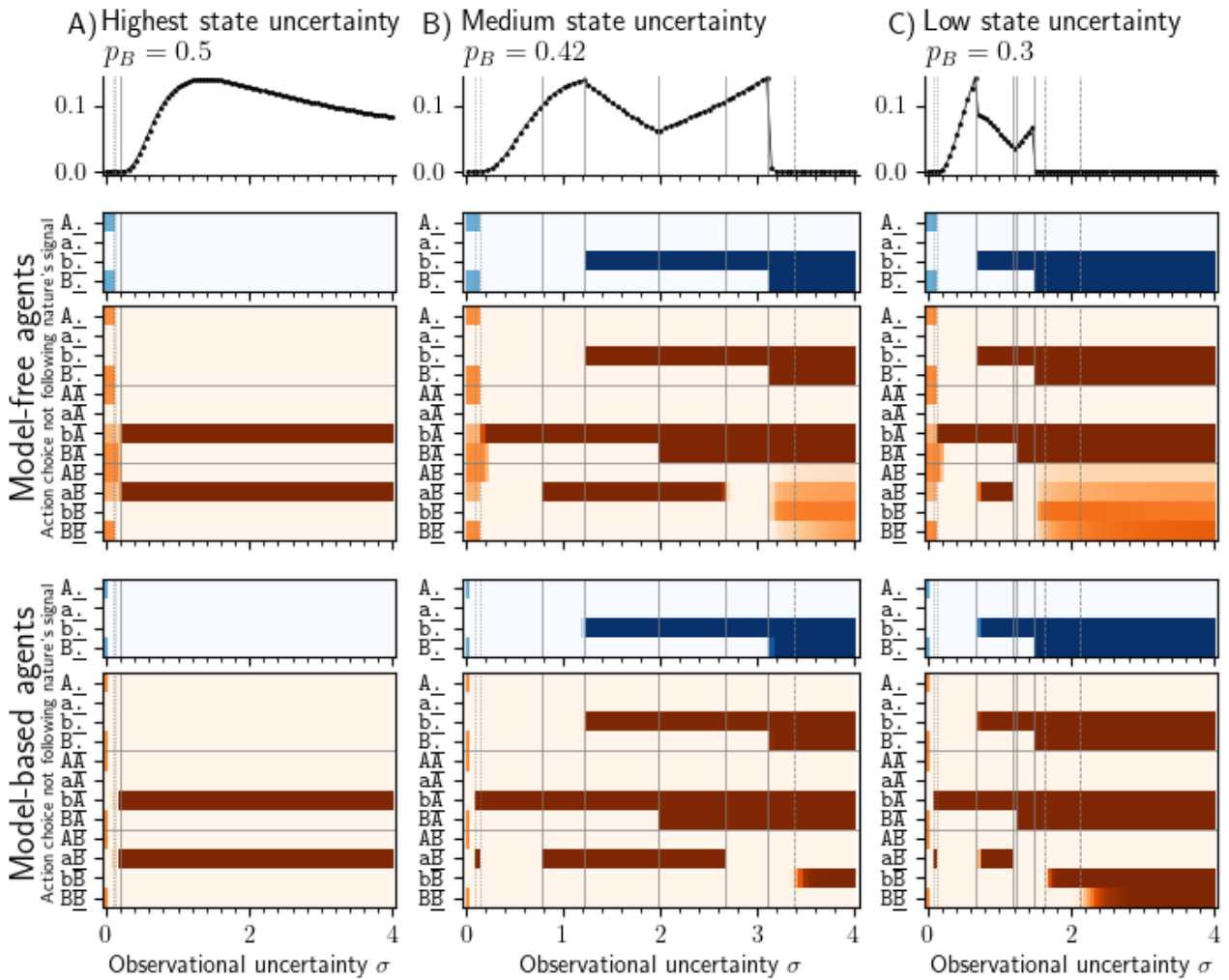


Figure 5.3: Figure 3 of the main text showing the learned strategies.

6 Result | Strategy convergence timescales

Here, we visualize the time steps required to reach the final strategies across different levels of environmental uncertainty. We start by importing the required packages, defining some configurations and obtaining the required simulation data.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
plt.rcParams['text.usetex'] = True
from matplotlib.gridspec import GridSpec
from matplotlib.lines import Line2D

from _code.ObtainNoiseVariationResults import
    ↪ obtain_all_noise_variation_comparisons, noise_levels
(NoiVar_MBA_pAhigh, NoiVar_MFA_pAhigh,
 NoiVar_MBA_pAmid, NoiVar_MFA_pAmid,
 NoiVar_MBA_pAlow, NoiVar_MFA_pAlow) = obtain_all_noise_variation_comparisons()
```

```
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initalX128421
    ↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.5_likelydistance0.5_LearningAgents_learning_rates0.25_initalX779
    ↪ 4212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initalX12631
    ↪ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.5_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initalX91
    ↪ 45693657658906017_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initalX12842
    ↪ 151645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initalX77
    ↪ 94212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initalX1263
    ↪ 1382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
    ↪ _probabilityA0.58_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initalX9
    ↪ 145693657658906017_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
    ↪ obabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initalX128421
    ↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
```

```

Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem_
↳ _probabilityA0.7_likelydistance0.5_LearningAgents_learning_rates0.25_initialX779
↳ 4212447693859915_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↳ obabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX12631
↳ 382026377322370_maxT250000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem_
↳ _probabilityA0.7_likelydistance0.5_ModelFreeAgents_learning_rates0.25_initialX91
↳ 45693657658906017_maxT250000_Noises7752999609125433264.npz

```

```

# Color palette (Okabe-Ito - colorblind-safe)
COLOR_SINGLE = "#0072B2" # blue - single-agent environment
COLOR_SOCIAL = "#D55E00" # vermillion - two-agent environment

```

We create a function to plot the time it takes for behaviors to converge under different noise levels and model assumptions.

```

def plot_noise_variation(observable_Onk, noise_levels,
                        quartiles_color, observable_range,
                        ax=None):

    quartile1, medians, quartile3 = np.percentile(
        observable_Onk, [5, 50, 95], axis=1)

    ax.fill_between(noise_levels, quartile1, quartile3,
                    color=quartiles_color, alpha=0.5)
    ax.plot(noise_levels, medians, '-', color=quartiles_color)
    ax.set_ylim(*observable_range)

```

This function is then used in another fuction that visualizes the comparison between the single-agent and the two-agent environments.

```

def plot_behavior_convergence(NoiVar, maxT, ax=None):
    if ax is None:
        ax = plt.gca()

    params = dict(
        noise_levels=noise_levels,
        # number_of_bins_for_observable=51,
        observable_range=(0, maxT),
        # plot_means=False,
        # plot_histogram=False,
        ax=ax,
    )

    plot_noise_variation(
        observable_Onk=NoiVar.sa_Tnk, quartiles_color=COLOR_SINGLE, **params
    )

    plot_noise_variation(
        observable_Onk=NoiVar.ta_Tnk, quartiles_color=COLOR_SOCIAL, **params
    )

```

```

xlim=[-0.05, max(noise_levels)+0.05]
ax.set_xlim(xlim)
ax.xaxis.set_minor_locator(MultipleLocator(0.2))
ax.set_xlabel('Observational uncertainty  $\sigma$ ')
ax.set_ylabel("Convergence steps")

```

This function yields, for example,

```

plot_behavior_convergence(NoiVar_MFA_pAllow, maxT=2500)
plt.ylim(100, 100000); plt.yscale("log")

```

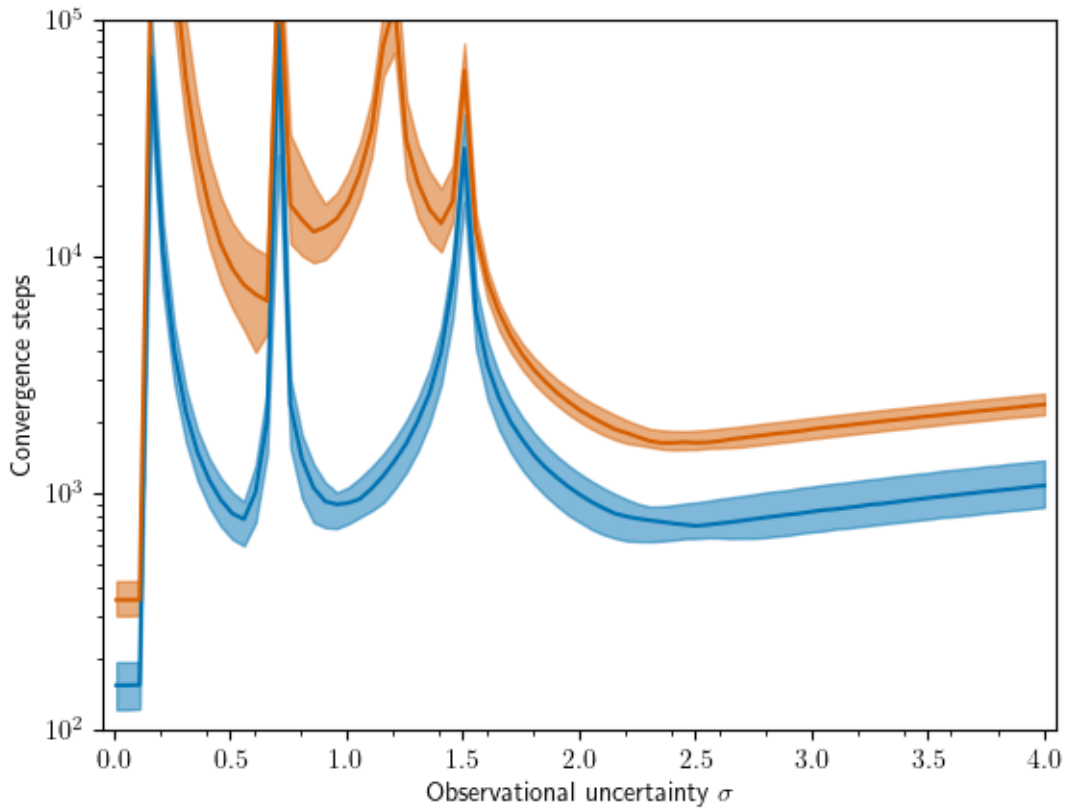


Figure 6.1: Example plot showing the time it takes for strategies to converge in the single-agent (blue) and two-agent (red) environments versus observational uncertainty.

Again, we define the two helper functions to plot vertical lines to better visualize the critical levels of observational uncertainty where strategy transitions occur.

```

def plot_vertical_line_at_noise_level(ax, noise_level, ls='-'):
    ax.plot([noise_level, noise_level], ax.get_ylim(), ls, color='gray', lw=1.0)

def plot_vertical_lines_at_all_axes(axes, noise_level, ls='-'):
    for ax in axes: plot_vertical_line_at_noise_level(ax, noise_level, ls)

```

Putting everything together, we create the Figure 4 of the main text showing the learning time steps required.


```

fsf = 0.260
fig = plt.figure(figsize=(fsf*32, fsf*16))

gs = GridSpec(2, 3, hspace=0.08, wspace=0.08,
              height_ratios=[1, 1], width_ratios=[1, 1.35, 1])

titleparams = dict(loc="left", x=0.12)
ABCparams = dict(xy=(0.0, 1.2), ha='left', va='top',
                  xycoords='axes fraction', fontsize=14, fontweight='bold')
maxT_MFA = 100000; maxT_MBA = 2500
MFAylim = (100, maxT_MFA); MBAylim = (10, maxT_MBA);

agentmodelparams = dict(
    xy=(-0.34, 0.5), rotation=90, fontsize=14, ha='center',
    va='center', xycoords='axes fraction', fontweight='bold')

## 1 | Highest state uncertainty (p(A)=0.5)
ax_MFA_high = fig.add_subplot(gs[0, 0]);
ax_MFA_high.set_title("Highest state uncertainty\n$p_B=0.5$", **titleparams)
ax_MFA_high.annotate("A", **ABCparams)
plot_behavior_convergence(NoiVar_MFA_pAhigh, maxT=maxT_MFA, ax=ax_MFA_high)
ax_MFA_high.set_xlabel(""); ax_MFA_high.set_xticklabels([])
ax_MFA_high.set_ylabel("Convergence steps")
ax_MFA_high.annotate("Model-free agents", **agentmodelparams)
ax_MFA_high.set_ylim(MFAylim); ax_MFA_high.set_yscale("log")
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

ax_MBA_high = fig.add_subplot(gs[1, 0]);
plot_behavior_convergence(NoiVar_MBA_pAhigh, maxT=maxT_MBA, ax=ax_MBA_high)
ax_MBA_high.annotate("Model-based agents", **agentmodelparams)
ax_MBA_high.set_ylim(MBAylim); ax_MBA_high.set_yscale("log")
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

axes = [ax_MFA_high, ax_MBA_high]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.19)

## 2 | Medium state uncertainty (p(A)=0.6)
ax_MFA_mid = fig.add_subplot(gs[0, 1]);
ax_MFA_mid.set_title("Medium state uncertainty\n$p_B=0.42$", **titleparams)
ax_MFA_mid.annotate("B", **ABCparams)
plot_behavior_convergence(NoiVar_MFA_pAamid, maxT=maxT_MFA, ax=ax_MFA_mid)
ax_MFA_mid.set_ylim(MFAylim); ax_MFA_mid.set_yscale("log")
ax_MFA_mid.set_ylabel(""); ax_MFA_mid.set_yticklabels([])
ax_MFA_mid.set_xlabel(""); ax_MFA_mid.set_xticklabels([])
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

ax_MBA_mid = fig.add_subplot(gs[1, 1]);
plot_behavior_convergence(NoiVar_MBA_pAamid, maxT=maxT_MBA, ax=ax_MBA_mid)
ax_MBA_mid.set_ylim(MBAylim); ax_MBA_mid.set_yscale("log")

```

```

ax_MBA_mid.set_ylabel(""); ax_MBA_mid.set_yticklabels([])
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

axes = [ax_MFA_mid, ax_MBA_mid]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.78)
plot_vertical_lines_at_all_axes(axes, 1.22)
plot_vertical_lines_at_all_axes(axes, 1.98)
plot_vertical_lines_at_all_axes(axes, 2.68)
plot_vertical_lines_at_all_axes(axes, 3.11)
plot_vertical_lines_at_all_axes(axes, 3.38, ls="--")

## 3 | Lowest state uncertainty (p(A)=0.7)
ax_MFA_low = fig.add_subplot(gs[0, 2]);
ax_MFA_low.set_title("Low state uncertainty\n$p_B=0.3$", **titleparams)
ax_MFA_low.annotate("C", **ABCparams)
plot_behavior_convergence(NoiVar_MFA_pA_low, maxT=maxT_MFA, ax=ax_MFA_low)
ax_MFA_low.set_ylim(MFAylim); ax_MFA_low.set_yscale("log")
ax_MFA_low.set_ylabel(""); ax_MFA_low.set_yticklabels([])
ax_MFA_low.set_xlabel(""); ax_MFA_low.set_xticklabels([])
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

ax_MBA_low = fig.add_subplot(gs[1, 2]);
plot_behavior_convergence(NoiVar_MBA_pA_low, maxT=maxT_MBA, ax=ax_MBA_low)
ax_MBA_low.set_ylim(MBAylim); ax_MBA_low.set_yscale("log")
ax_MBA_low.set_ylabel(""); ax_MBA_low.set_yticklabels([]);
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

axes = [ax_MFA_low, ax_MBA_low]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.68)
plot_vertical_lines_at_all_axes(axes, 1.18)
plot_vertical_lines_at_all_axes(axes, 1.23)
plot_vertical_lines_at_all_axes(axes, 1.48)
plot_vertical_lines_at_all_axes(axes, 1.63, ls="--")
plot_vertical_lines_at_all_axes(axes, 2.13, ls="--")

# Legend
legend_elements = [Line2D([0], [0], color=COLOR_SINGLE, lw=2, label='Single agent'),
                   Line2D([0], [0], color=COLOR_SOCIAL, lw=2, label='Two agents')]
ax_MFA_high.legend(handles=legend_elements, loc='lower right')

plt.savefig("./_figs/BehaviorConvergence.png", bbox_inches='tight', dpi=300)

```

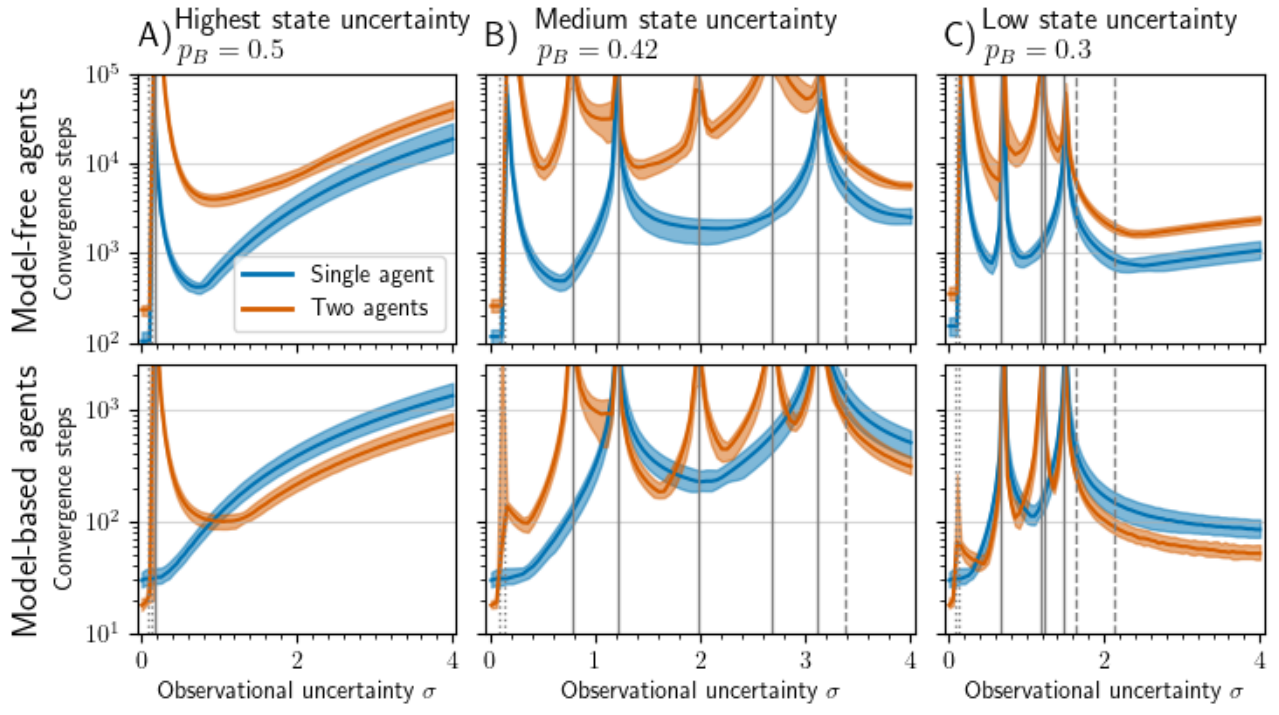


Figure 6.2: Figure 4 of the main text showing the learning times.

7 Result | Reward performance timescales

Here, we compute and visualize the timescales required to reach almost the rewards of the final strategies across different levels of environmental uncertainty. We start by importing the required packages, defining some configurations and obtaining the required simulation data.

```
import pickle
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
from matplotlib.gridspec import GridSpec
from matplotlib.lines import Line2D
plt.rcParams['text.usetex'] = True

from collections import namedtuple

from _code.UncertainDecisionProblem import \
    SingleAgentUncertainDecisionProblem, TwoAgentUncertainDecisionProblem

from _code.LearningDynamics import LearningAgents, ModelFreeAgents
from _code.SimulationScripts import _transform_tensor_into_hash
from _code.ObtainNoiseVariationResults import lowlevel_datadictionary, noise_levels

# Color palette (Okabe-Ito - colorblind-safe)
COLOR_SINGLE = "#0072B2" # blue - single-agent environment
COLOR_SOCIAL = "#D55E00" # vermillion - two-agent environment
COLOR_ACROSS = "#009E73" # bluish green - time for two agents to match single-agent
    ↪ performance

Aprobabilities = [0.5, 0.52, 0.55, 0.58, 0.6, 0.7, 0.9]
```

Compute reward trajectory function

We start by creating a function to compute trajectories that show how the average reward evolves over time, starting from a uniformly random initial strategy.

```
def compute_reward_trajectory(AgentClass, EnvClass,
                              NoiseLevel, probabilityA=0.5,
                              maxT=100000, verbose=True):

    envparams = dict(noiselevel=NoiseLevel,
                     probabilityA=probabilityA,
                     likelydistance=0.5)
```

```

if verbose:
    print(f"{AgentClass.__name__} in {EnvClass.__name__}"
          f" with Noiselevel: {NoiseLevel:.5f}", end=" ")

env = EnvClass(**envparams)
maei = AgentClass(env, learning_rates=0.25)

Xioa = maei.zero_intelligence_policy()
Xtioa, fpr = maei.trajectory(Xioa, Tmax=maxT, tolerance=10e-7)
if verbose:
    print(f"FPR: {fpr}", end="\r")
Rti = np.array([maei.Ri(Xioa) for Xioa in Xtioa])

result = namedtuple("Result", ["Rti", "Xtioa", "fpr"])
return result(Rti=Rti, Xtioa=Xtioa, fpr=fpr)

```

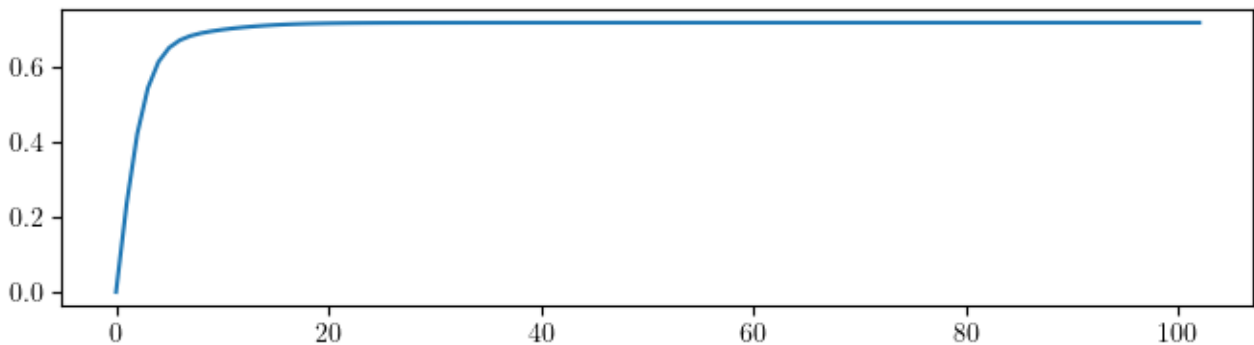
For example,

```

Rt = compute_reward_trajectory(LearningAgents, TwoAgentUncertainDecisionProblem,
                               NoiseLevel=0.5, probabilityA=0.6)
fig = plt.figure(figsize=(8,2)); plt.plot(Rt.Rti.mean(-1));

```

LearningAgents in TwoAgentUncertainDecisionProblem with Noiselevel: 0.50000 FPR: True



Compute reward trajectories over noise levels

Now, we create a function to compute the reward trajectories over different levels of observational uncertainty for both the single-agent and two-agent environments.

```

def compute_reward_trajectories_over_noiselevels(
    AgentClass, EnvClass, NoiseLevels, probabilityA=0.5,
    maxT=100000):
    results = []
    for i, nl in enumerate(NoiseLevels):
        print(f"Computing for NoiseLevel={nl:.4f}"
              f" ({(i+1)/len(NoiseLevels)*100:.1f}%)", end="\r")
        res = compute_reward_trajectory(
            AgentClass, EnvClass, nl, probabilityA=probabilityA, maxT=maxT,
            verbose=False)

```

```

        results.append(res.Rti)
    return results

```

The `obtain_` function checks if the required data files already exist in the data directory. If they do, it loads them; otherwise, it computes the reward trajectories using the previously defined function and saves them for future use.

```

def obtain_reward_trajectories_over_noiselevels(
    AgentClass, EnvClass, NoiseLevels, ddir,
    probabilityA=0.5, maxT=100000):

    ## Create filename
    fn = (ddir +
          f"/RewardNOISVARI_"
          f"{AgentClass.__name__}_"
          f"{EnvClass.__name__}_"
          f"pA{probabilityA}_"
          f"Noises{_transform_tensor_into_hash(np.array(NoiseLevels))}_"
          f"maxT{maxT}"
          f".pkl")

    try:
        with open(fn, "rb") as f: Rnti = pickle.load(f)
        print("Loaded reward noise variation:", fn)

    except:
        print("Computing noise variation:", fn)
        Rnti = compute_reward_trajectories_over_noiselevels(
            AgentClass, EnvClass, NoiseLevels,
            probabilityA=probabilityA, maxT=maxT)

        with open(fn, "wb") as f: pickle.dump(Rnti, f)
        with open(fn, "rb") as f: Rnti = pickle.load(f)

    return Rnti

```

Heavy computation

The following code cell obtains (i.e., computes or loads) all the required reward trajectory data for both the single-agent and two-agent environments across different levels of observational uncertainty.

```

Rnti = dict()
for Akey, AgentClass in zip(["MBA", "MFA"], [LearningAgents, ModelFreeAgents]):
    for pA in Aprobabilities:
        for Ekey, EnvClass in zip(["TAE", "SAE"], [TwoAgentUncertainDecisionProblem,
            ↪ SingleAgentUncertainDecisionProblem]):

            individual_Rnti = obtain_reward_trajectories_over_noiselevels(
                AgentClass, EnvClass, noise_levels,
                probabilityA=pA, ddir=lowlevel_datadictionary)

            Rnti[(Akey, Ekey, pA)] = individual_Rnti

```

Data loaded from disk.

Results

We are finally ready to visualize the timescales required to reach almost the rewards of the final strategies across different levels of environmental uncertainty. We start by defining a function to plot an overview how the two different learning dynamics compare across uncertainty levels.

```
def overview_plot(plot_func, ylabel,
                  MFAylim=(None, None), MBAYlim=(None, None), **pfunc_kwargs):
    fsf = 0.50

    nr_pA = len(Aprobabilities)
    fig, axes = plt.subplots(nr_pA, 2, figsize=(fsf*16, fsf*20))

    for i, axs in enumerate(axes):
        pA = Aprobabilities[i]

        # # data plotting
        plot_func(
            Rnti, noise_levels, Akey="MFA", pA=pA, ax=axs[0], **pfunc_kwargs)
        plot_func(
            Rnti, noise_levels, Akey="MBA", pA=pA, ax=axs[1], **pfunc_kwargs)

        su = np.round(pA, 2)
        axs[0].set_ylabel(f"$p_B={1-su:.2f}$"); axs[1].set_ylabel("");
        axs[0].set_ylim(MFAylim), axs[1].set_ylim(MBAYlim)

        axs[0].xaxis.set_minor_locator(MultipleLocator(0.2))
        axs[1].xaxis.set_minor_locator(MultipleLocator(0.2))
        axs[0].grid(True, which='major', axis='both', alpha=0.5)
        axs[1].grid(True, which='major', axis='both', alpha=0.5)

    for i in range(nr_pA-1):
        axes[i,0].set_xticklabels(""); axes[i,0].set_xlabel("")
        axes[i,1].set_xticklabels(""); axes[i,1].set_xlabel("")

    # Add global y label
    plt.gca().annotate(ylabel, (0.01, 0.45),
                       xycoords='figure fraction',
                       rotation=90, ha='left', va='center', fontsize=13)
    axes[0,0].set_title("Model-Free Agents", fontsize=13)
    axes[0,1].set_title("Model-Based Agents", fontsize=13);

    plt.subplots_adjust(hspace=0.08)
```

Percentage length

First, we define a function that returns the number of time steps required to reach a certain percentage of the final reward.

```
def find_percentage_length(Rti, percentage):
    final_reward = Rti[-1].mean()
    threshold = percentage * final_reward
    try:
        length = min(np.where(Rti.mean(-1) >= threshold)[0])
    except ValueError:
        length = len(Rti)
    return length
```

This function is then used to visualize how quickly the different learning dynamics reach various percentages of the final reward across different levels of environmental uncertainty.

```
def plot_percentage_convergence_lengths(Rnti, noise_levels, ax=None,
                                       Akey="MBA", pA=0.5,
                                       percentages=[0.9, 0.99, 0.999],
                                       ylogscale=False):

    if ax is None: ax = plt.gca()

    TAE_lengths = []
    SAE_lengths = []
    for perc in percentages:
        TAE_lengths.append([find_percentage_length(Rti, perc)
                           for Rti in Rnti[(Akey, "TAE", pA)]])
        SAE_lengths.append([find_percentage_length(Rti, perc)
                           for Rti in Rnti[(Akey, "SAE", pA)]])

    ax.fill_between(noise_levels, SAE_lengths[0], SAE_lengths[-1],
        ↪ color=COLOR_SINGLE,
        alpha=0.2, label="Single-Agent range")
    ax.fill_between(noise_levels, TAE_lengths[0], TAE_lengths[-1],
    ↪ color=COLOR_SOCIAL,
        alpha=0.2, label="Two-Agent range")

    # # Plot the highest percentage lines for reference
    # percentage = percentages[-1]
    ax.plot(noise_levels, SAE_lengths[1], c=COLOR_SINGLE)
    ax.plot(noise_levels, TAE_lengths[1], c=COLOR_SOCIAL)
    ax.set_xlabel('Observational uncertainty  $\sigma$ ')
    # ax.set_ylabel(f"Reward Convergence Length to {percentage*100:.0f}% of final
    ↪ reward (timesteps)")
    xlim=[-0.05, max(noise_levels)+0.05]
    ax.set_xlim(xlim)
    if ylogscale: ax.set_yscale("log")
    ax.grid(True, which='major', axis='both', alpha=0.5)
```

For example,

```
plot_percentage_convergence_lengths(
    Rnti, noise_levels, Akey="MBA", pA=0.6, percentages=[0.925, 0.95, 0.975])
plt.ylim(3, 200);
```

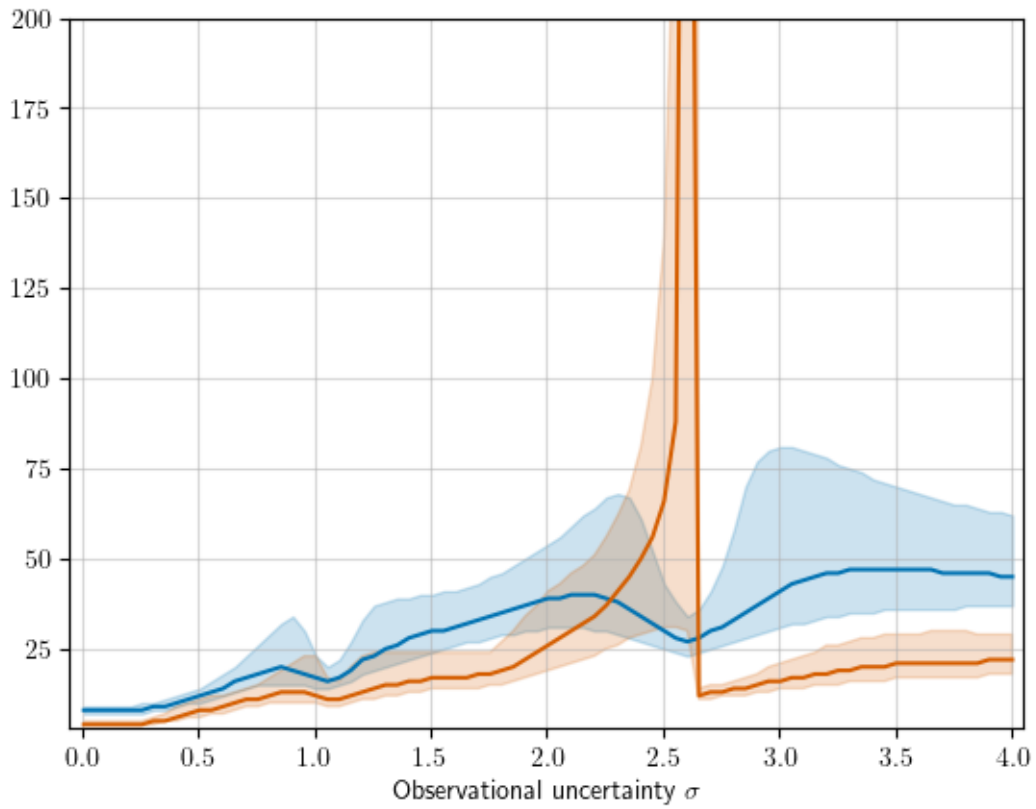



Figure 7.1: Example of reward-percentage convergence lengths visualization.

Inserting this visualization function into the overview plot yields,

```
overview_plot(plot_percentage_convergence_lengths,
              ylabel="Reward Percentage Convergence Length [time steps]",
              MFAylim=(30, 10000),
              MBAYlim=(3, 1000),
              percentages=[0.97, 0.98, 0.99],
              ylogscale=True
            )
```

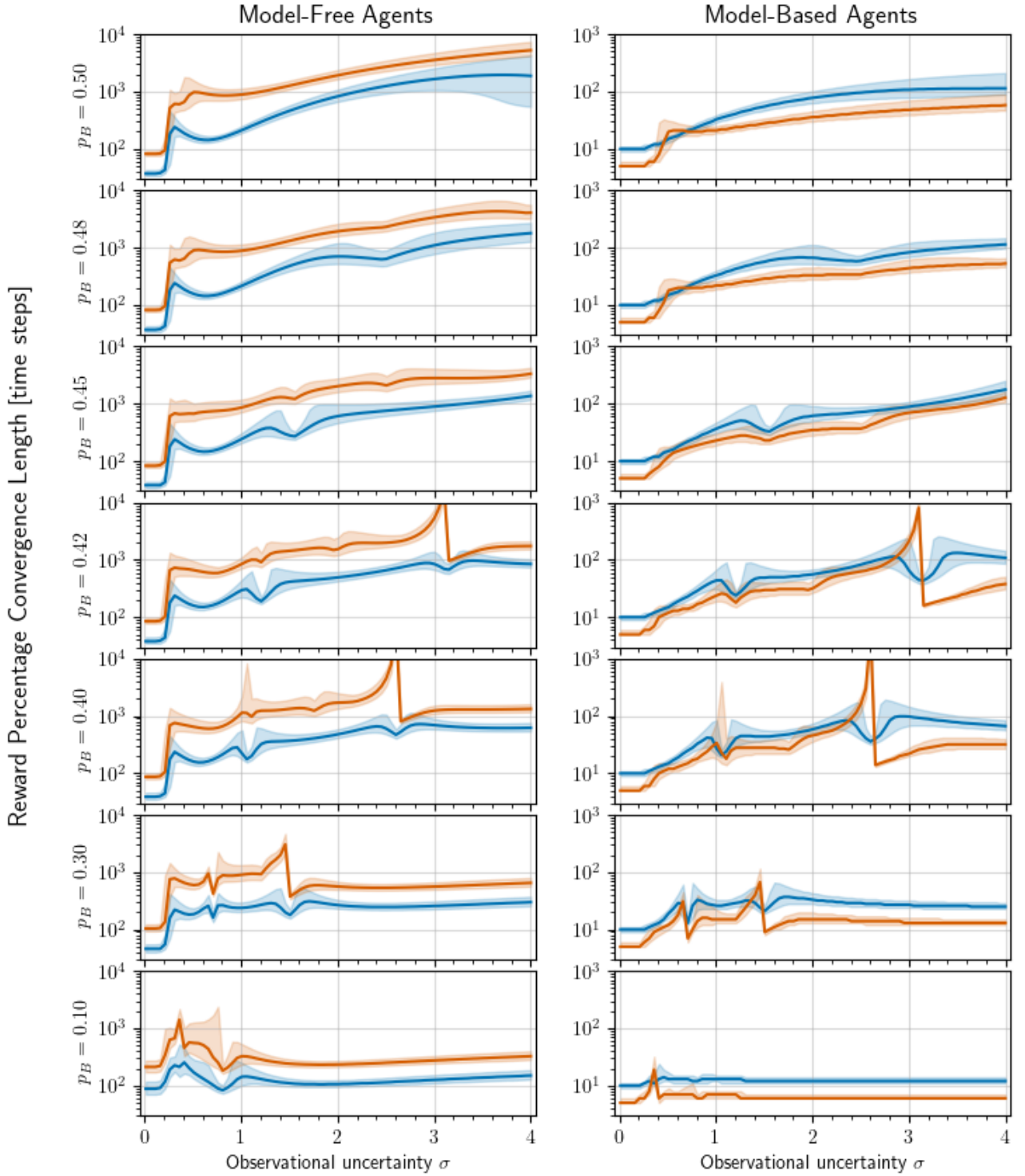


Figure 7.2: Overview of the learning time steps required until 98% ($\pm 1\%$) of the final reward at strategy convergence is reached versus observational uncertainty σ for all state uncertainties p_B investigated. The left panels show model-free learning dynamics, the right panels show model-based ones.

Compared performance length

We also visualize the time steps required for the two-agent learners to reach the respective performance levels of the single-agent learner. To do so, we define a function that computes the time steps required to reach a specific threshold reward value.

```
def find_comparison_length(Rti, threshold):
    try:
        length = min(np.where(Rti.mean(-1) >= threshold)[0])
    except ValueError:
        length = np.inf
    return length
```

We insert the above function into an explicit comparison function that visualizes the time steps required for the two-agent learners to reach the respective performance levels of the single-agent learner,

```
def plot_compared_percentage_convergence_lengths(
    Rnti, noise_levels, ax=None, Akey="MBA", pA=0.5,
    percentages=[0.9, 0.99, 0.999], ylogscale=False):
    if ax is None: ax = plt.gca()

    lengthss = []

    for perc in percentages:

        lengths = []
        for n, Rti in enumerate(Rnti[(Akey, "TAE", pA)]):
            threshold = perc*Rnti[(Akey, "SAE", pA)][n][-1].mean(-1)
            lengths.append(find_comparison_length(Rti, threshold))

        lengthss.append(lengths)

    ax.fill_between(noise_levels, lengthss[0], lengthss[-1], color=COLOR_ACROSS,
                    alpha=0.2)

    ax.plot(noise_levels, lengthss[1], c=COLOR_ACROSS)
    ax.set_xlabel('Observational uncertainty  $\sigma$ ')
    # ax.set_ylabel(f"Reward Convergence Length to {percentage*100:.0f}% of final
    #   ↪ reward (timesteps)")
    xlim=[-0.05, max(noise_levels)+0.05]
    ax.set_xlim(xlim)
    if ylogscale: ax.set_yscale("log")
```

This function yields, for example,

```
plot_compared_percentage_convergence_lengths(
    Rnti, noise_levels, Akey="MBA", pA=0.5, percentages=[0.95, 0.99, 1.0])
```

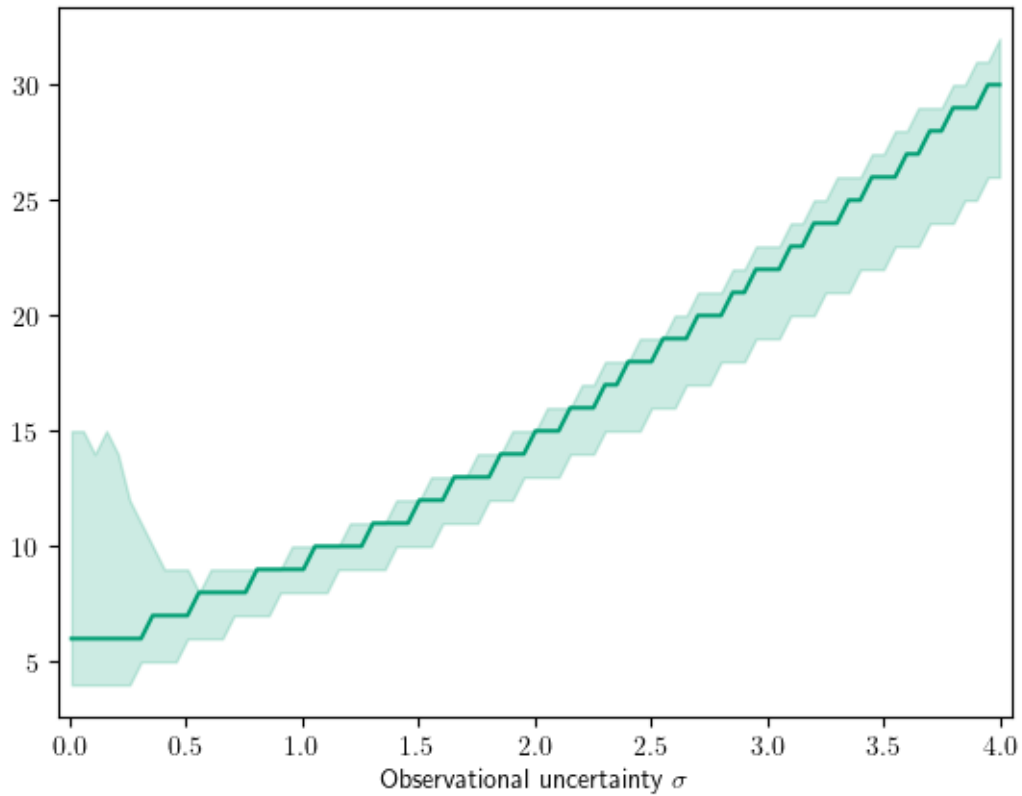


Figure 7.3: Example of compared reward-percentage convergence lengths visualization.

and across all state uncertainties and agent models,

```
overview_plot(plot_compared_percentage_convergence_lengths,
              ylabel="Reward Percentage Comparison Convergence Length [time steps]",
              MFAylim=(30, 1500),
              MBAylim=(3, 30),
              percentages=[0.97, 0.98, 0.99],
              # ylogscale=True
              )
```

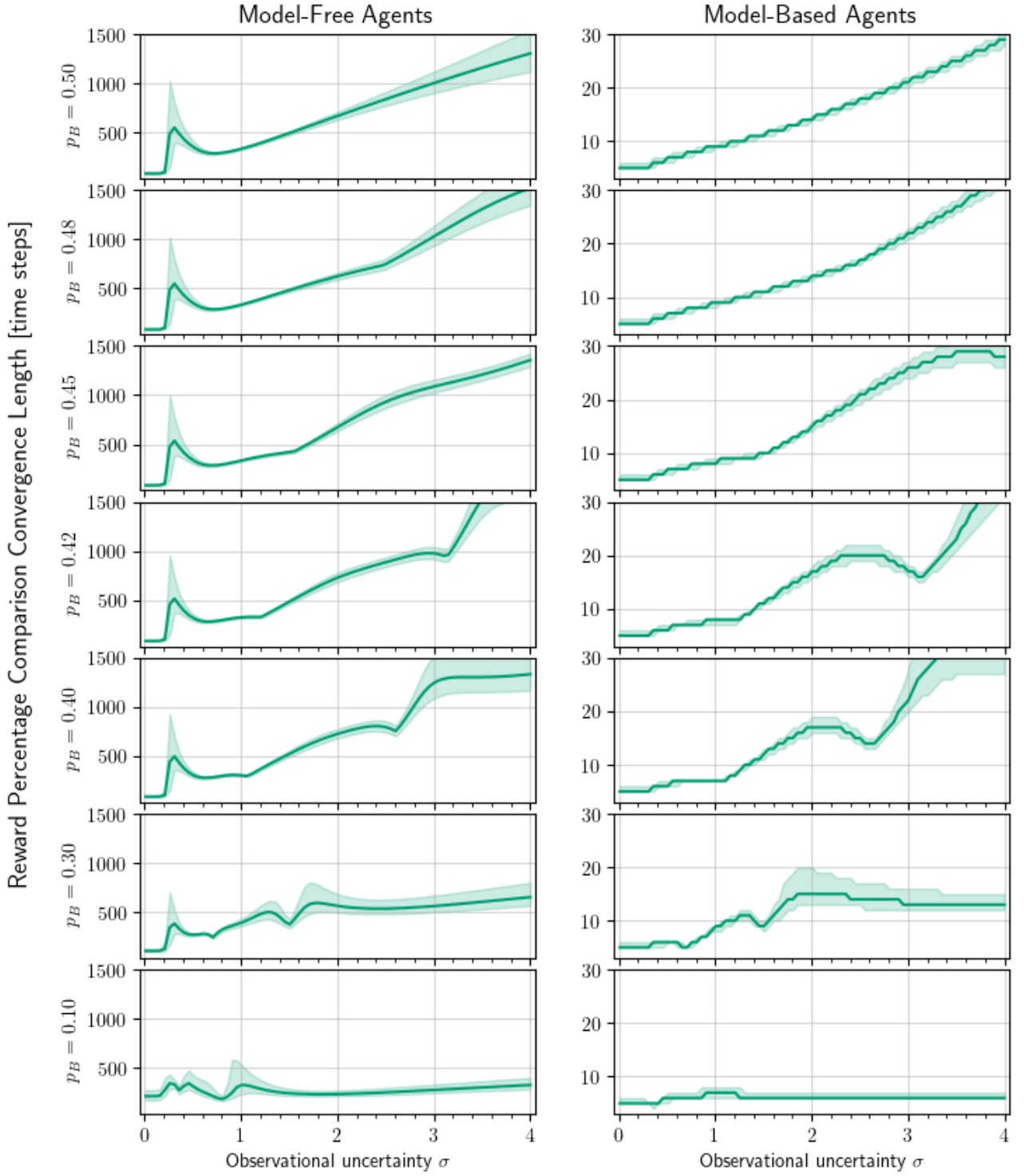


Figure 7.4: Overview of the learning time steps required until 98% ($\pm 1\%$) of the final reward of the single-agent learner at strategy convergence is reached by the two-agent learners versus observational uncertainty σ for all state uncertainties p_B investigated. The left panels show model-free learning dynamics, the right panels show model-based ones.

Combined percentance lengths

Last, we create a function to plot both measures,

```

def plot_percentage_comparison_convergence_lengths(Rnti, noise_levels, ax=None,
                                                  Akey="MBA", pA=0.5,
                                                  percentages=[0.9, 0.99, 0.999],
                                                  ylogscale=False):

    if ax is None: ax = plt.gca()

    TAE_lengths = []
    SAE_lengths = []
    com_lengths = []
    for perc in percentages:

        com_lens = []
        for n, Rti in enumerate(Rnti[(Akey, "TAE", pA)]):
            threshold = perc*Rnti[(Akey, "SAE", pA)][n][-1].mean(-1)
            com_lens.append(find_comparison_length(Rti, threshold))
        com_lengths.append(com_lens)

        TAE_lengths.append([find_percentage_length(Rti, perc)
                             for Rti in Rnti[(Akey, "TAE", pA)]])

        SAE_lengths.append([find_percentage_length(Rti, perc)
                             for Rti in Rnti[(Akey, "SAE", pA)]])

    ax.fill_between(noise_levels, SAE_lengths[0], SAE_lengths[-1],
                    color=COLOR_SINGLE,
                    alpha=0.2, label="Single-Agent range")
    ax.fill_between(noise_levels, TAE_lengths[0], TAE_lengths[-1],
                    color=COLOR_SOCIAL,
                    alpha=0.2, label="Two-Agent range")

    ax.fill_between(noise_levels, com_lengths[0], com_lengths[-1],
                    color=COLOR_ACROSS,
                    alpha=0.2)

    # # Plot the highest percentage lines for reference
    # percentage = percentages[-1]
    ax.plot(noise_levels, SAE_lengths[1], c=COLOR_SINGLE)
    ax.plot(noise_levels, TAE_lengths[1], c=COLOR_SOCIAL)
    ax.plot(noise_levels, com_lengths[1], c=COLOR_ACROSS)

    ax.set_xlabel('Observational uncertainty  $\sigma$ ')
    # ax.set_ylabel(f"Reward Convergence Length to {percentage*100:.0f}% of final
    # ↪ reward (timesteps)")
    xlim=[-0.05, max(noise_levels)+0.05]
    ax.set_xlim(xlim)
    if ylogscale: ax.set_yscale("log")
    ax.xaxis.set_minor_locator(MultipleLocator(0.2))

```

It yields, for example,

```

plot_percentage_comparison_convergence_lengths(
    Rnti, noise_levels, Akey="MFA", pA=0.5, percentages=[0.97, 0.98, 0.99])

```

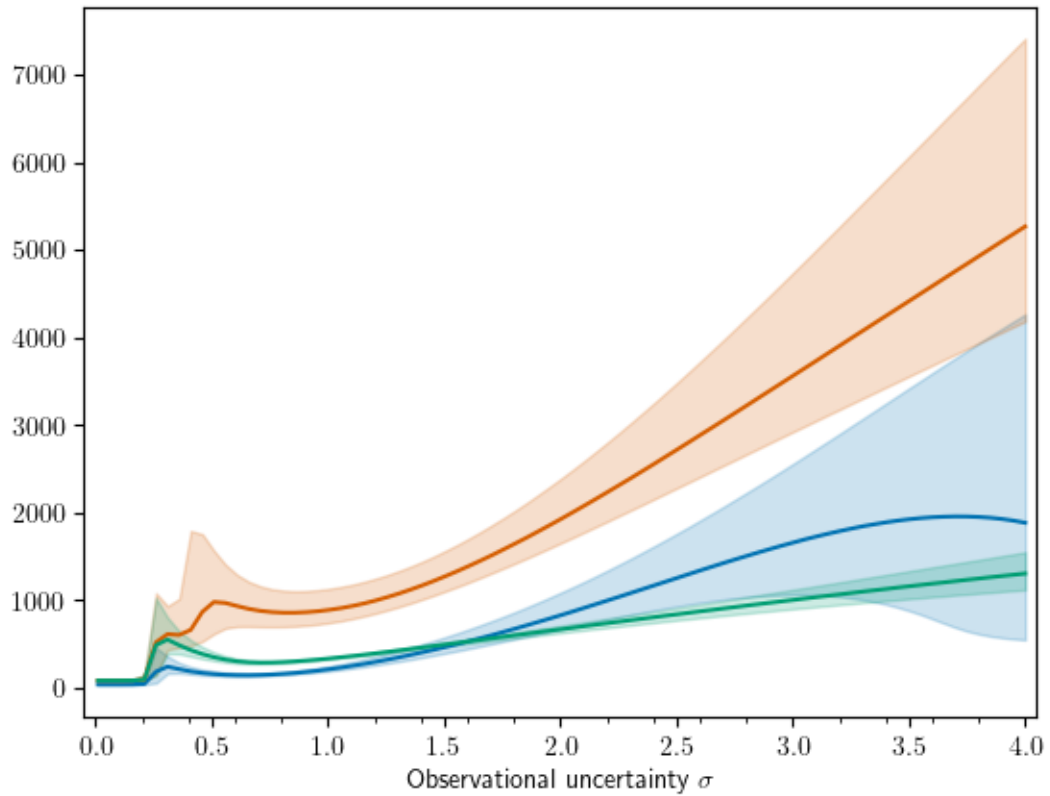


Figure 7.5: Example of combined reward-percentage convergence lengths visualization.

and across all state uncertainties and agent models,

```
overview_plot(plot_percentage_comparison_convergence_lengths,
              ylabel="Reward Percentage Convergence Length [time steps]",
              MFAylim=(30, 10000),
              MBAylim=(3, 1000),
              percentages=[0.97, 0.98, 0.99],
              ylogscale=True
            )
```

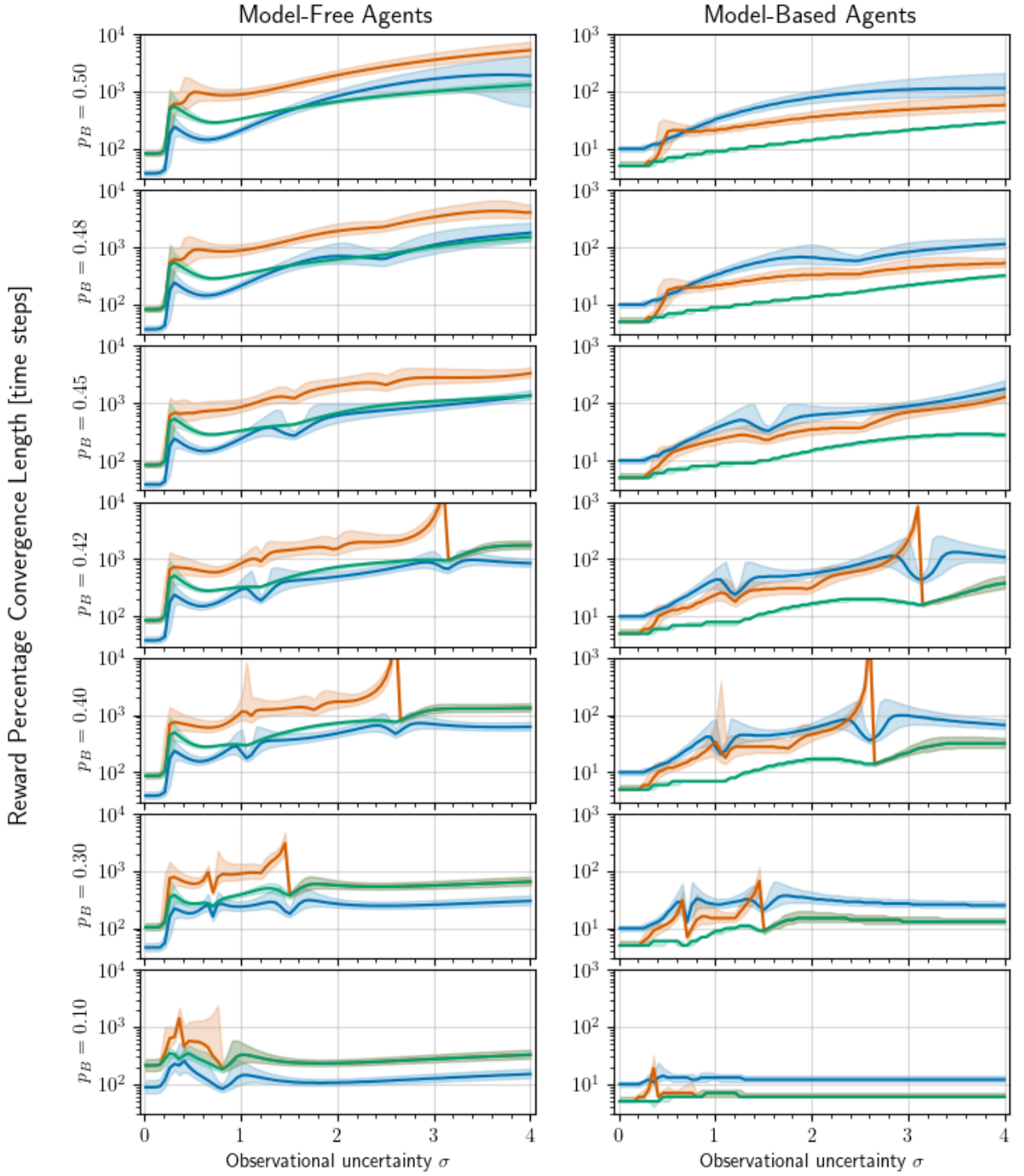


Figure 7.6: Overview of the learning time steps required until 98% ($\pm 1\%$) of the final reward versus observational uncertainty σ for all state uncertainties p_B investigated. The left panels show model-free learning dynamics, the right panels show model-based ones. Blue colors indicate the single-agent environment, vermilion colors indicate the two-agent environment. The green color indicates the time steps required for the two learners to reach the respective performance level of the single agent.

Main text figure

Again, we define the two helper functions to plot vertical lines to better visualize the critical levels of observational uncertainty where strategy transitions occur.

```
def plot_vertical_line_at_noise_level(ax, noise_level, ls='-'):  
    ax.plot([noise_level, noise_level], ax.get_ylim(), ls, color='gray', lw=1.0)
```

```
def plot_vertical_lines_at_all_axes(axes, noise_level, ls='-'):  
    for ax in axes: plot_vertical_line_at_noise_level(ax, noise_level, ls)
```

Putting everything together, we create the Figure 5 of the main text showing the learning time steps required to reach almost the rewards of the final strategies across different levels of environmental uncertainty.

```
fsf = 0.26  
fig = plt.figure(figsize=(fsf*32, fsf*16))  
  
gs = GridSpec(2, 3, hspace=0.08, wspace=0.08,  
              height_ratios=[1, 1], width_ratios=[1, 1.35, 1])  
  
titleparams = dict(loc="left", x=0.11)  
ABCparams = dict(xy=(0.0, 1.2), ha='left', va='top',  
                 xycoords='axes fraction', fontsize=14, fontweight='bold')  
agentmodelparams = dict(  
    xy=(-0.42, 0.5), rotation=90, fontsize=14, ha='center',  
    va='center', xycoords='axes fraction', fontweight='bold')  
plotparams = dict(percentages=[0.97, 0.98, 0.99], ylogscale=True)  
MFAylim = (30, 10000); MBAylim = (3, 250)  
  
## 1 | Highest state uncertainty (p(A)=0.5)  
ax_MFA_high = fig.add_subplot(gs[0, 0]);  
ax_MFA_high.set_title("Highest state uncertainty\n$p_B=0.5$", **titleparams)  
ax_MFA_high.annotate("A)", **ABCparams)  
plot_percentage_comparison_convergence_lengths(  
    Rnti, noise_levels, Akey="MFA", pA=0.5, **plotparams)  
ax_MFA_high.set_xlabel(""); ax_MFA_high.set_xticklabels([])  
ax_MFA_high.set_ylabel("Steps to 98\\% ($\\pm$ 1\\%)\nPerformance")  
ax_MFA_high.annotate("Model-free agents", **agentmodelparams)  
ax_MFA_high.set_ylim(MFAylim)  
plt.gca().grid(True, which='major', axis='y', alpha=0.5)  
  
ax_MBA_high = fig.add_subplot(gs[1, 0]);  
plot_percentage_comparison_convergence_lengths(  
    Rnti, noise_levels, Akey="MBA", pA=0.5, **plotparams)  
ax_MBA_high.annotate("Model-based agents", **agentmodelparams)  
ax_MBA_high.set_ylim(MBAylim)  
ax_MBA_high.set_ylabel("Steps to 98\\% ($\\pm$ 1\\%)\nPerformance")  
plt.gca().grid(True, which='major', axis='y', alpha=0.5)  
  
axes = [ax_MFA_high, ax_MBA_high]  
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
```

```

plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.19)

## 2 | Medium state uncertainty (p(A)=0.6)
ax_MFA_mid = fig.add_subplot(gs[0, 1]);
ax_MFA_mid.set_title("Medium state uncertainty\n$p_B=0.42$", **titleparams)
ax_MFA_mid.annotate("B)", **ABCparams)
plot_percentage_comparison_convergence_lengths(
    Rnti, noise_levels, Akey="MFA", pA=0.58, **plotparams)
ax_MFA_mid.set_ylabel(""); ax_MFA_mid.set_yticklabels([])
ax_MFA_mid.set_xlabel(""); ax_MFA_mid.set_xticklabels([])
ax_MFA_mid.set_ylim(MFAylim)
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

ax_MBA_mid = fig.add_subplot(gs[1, 1]);
plot_percentage_comparison_convergence_lengths(
    Rnti, noise_levels, Akey="MBA", pA=0.58, **plotparams)
ax_MBA_mid.set_ylabel(""); ax_MBA_mid.set_yticklabels([])
ax_MBA_mid.set_ylim(MBAYlim)
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

axes = [ax_MFA_mid, ax_MBA_mid]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.78)
plot_vertical_lines_at_all_axes(axes, 1.22)
plot_vertical_lines_at_all_axes(axes, 1.98)
plot_vertical_lines_at_all_axes(axes, 2.68)
plot_vertical_lines_at_all_axes(axes, 3.11)
plot_vertical_lines_at_all_axes(axes, 3.38, ls="--")

## 3 | Lowest state uncertainty (p(A)=0.7)
ax_MFA_low = fig.add_subplot(gs[0, 2]);
ax_MFA_low.set_title("Low state uncertainty\n$p_B=0.3$", **titleparams)
ax_MFA_low.annotate("C)", **ABCparams)
plot_percentage_comparison_convergence_lengths(
    Rnti, noise_levels, Akey="MFA", pA=0.7, **plotparams)
ax_MFA_low.set_ylabel(""); ax_MFA_low.set_yticklabels([])
ax_MFA_low.set_xlabel(""); ax_MFA_low.set_xticklabels([])
ax_MFA_low.set_ylim(MFAylim)
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

ax_MBA_low = fig.add_subplot(gs[1, 2]);
plot_percentage_comparison_convergence_lengths(
    Rnti, noise_levels, Akey="MBA", pA=0.7, **plotparams)
ax_MBA_low.set_ylabel(""); ax_MBA_low.set_yticklabels([]);
ax_MBA_low.set_ylim(MBAYlim)
plt.gca().grid(True, which='major', axis='y', alpha=0.5)

axes = [ax_MFA_low, ax_MBA_low]
plot_vertical_lines_at_all_axes(axes, 0.09, ls=':')
plot_vertical_lines_at_all_axes(axes, 0.13, ls=':')

```

```

plot_vertical_lines_at_all_axes(axes, 0.68)
plot_vertical_lines_at_all_axes(axes, 1.18)
plot_vertical_lines_at_all_axes(axes, 1.23)
plot_vertical_lines_at_all_axes(axes, 1.48)
plot_vertical_lines_at_all_axes(axes, 1.63, ls="--")
plot_vertical_lines_at_all_axes(axes, 2.13, ls="--")

# Legend
legend_elements = [Line2D([0], [0], color=COLOR_SINGLE, lw=2, label='Single agent'),
                   Line2D([0], [0], color=COLOR_SOCIAL, lw=2, label='Two agents'),
                   Line2D([0], [0], color=COLOR_ACROSS, lw=2, label='Across')]
ax_MFA_high.legend(handles=legend_elements, loc='lower right')

plt.savefig("./_figs/RewardPerformanceSteps.png", bbox_inches='tight', dpi=300)

```

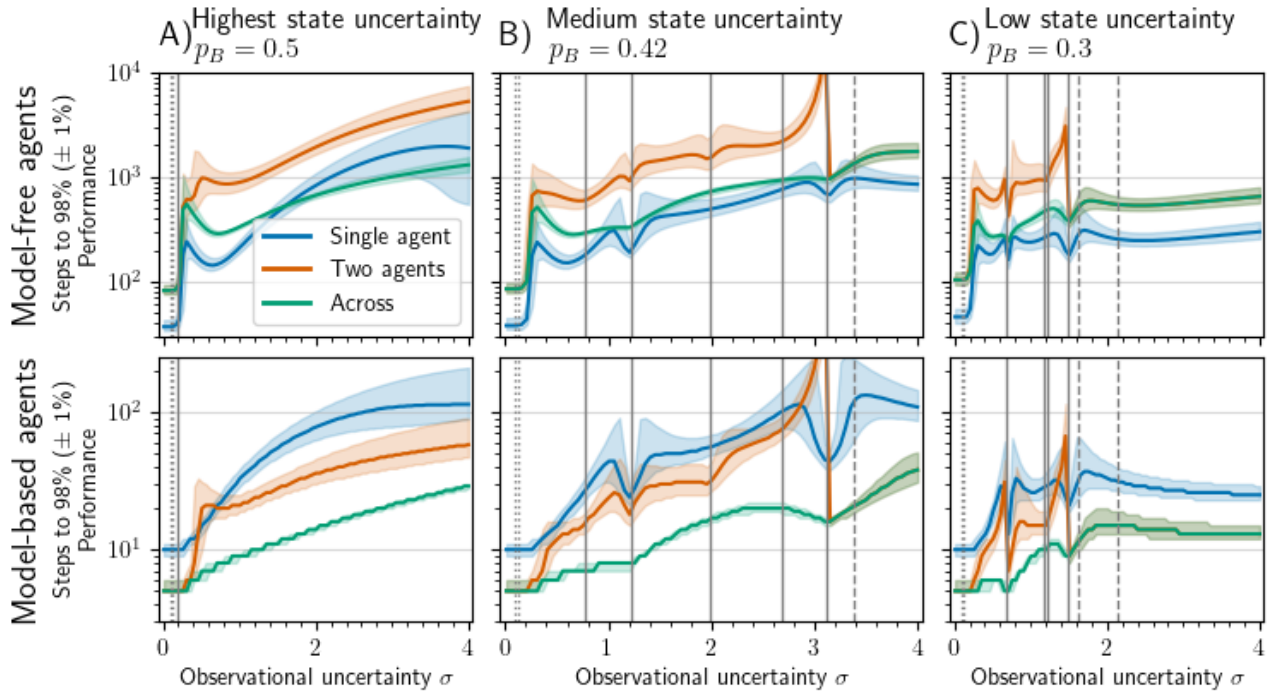


Figure 7.7: Figure 5 of the main text showing the learning time steps required to reach almost the rewards of the final strategies.

8 Auxiliary | Robustness checks

We verify that the key qualitative finding — social learning (two-agent model) converges faster than individual learning (single-agent model), with the speedup growing with observational uncertainty — is robust to the choice of learning rate α and convergence threshold ε .

All checks use the medium state uncertainty case ($p_A = 0.58$, $p_B = 0.42$), which shows the richest dynamics including multiple critical transitions. Robustness of this case implies robustness for higher and lower state uncertainty values.

We focus on model-based agents (deterministic replicator dynamics), which are the primary model of the paper.

```
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.ticker import MultipleLocator
from matplotlib.lines import Line2D
from collections import namedtuple

from _code.UncertainDecisionProblem import (
    SingleAgentUncertainDecisionProblem, TwoAgentUncertainDecisionProblem)
from _code.LearningDynamics import LearningAgents
from _code.SimulationScripts import (
    create_consistent_linspace, create_initial_lhs,
    compute_trajectories, obtain_noise_variation,
    final_rewards_Rki, collective_intelligence)
```

Shared parameters

We use the same observational uncertainty grid as the main results. For the convergence-threshold check we additionally use a coarser grid of 21 points (no caching layer is applied there, so fewer points keeps runtime manageable).

```
lowlevel_datadictionary = "/Users/wolf/Downloads/L2SocialData"
highlevel_datadictionary = "./_data"
ddirs = dict(lowlevel_ddir=lowlevel_datadictionary,
             highlevel_ddir=highlevel_datadictionary)

probabilityA = 0.58 # medium state uncertainty: p_B = 0.42
likelydistance = 0.5

noise_levels = create_consistent_linspace(0.01, 4.0, 81) # full grid (
    ↪ check)
noise_levels_coarse = create_consistent_linspace(0.01, 4.0, 21) # coarse grid (
    ↪ check)
```

```

alpha_values = [0.125, 0.25, 0.5]      # learning rates; baseline = 0.25
epsilon_values = [10e-6, 10e-7, 10e-8] # convergence thresholds; baseline =
↳ 1e-6 (= 10e-7 in code)

```

Helper: plot convergence-time curve

```

COLOR_SINGLE = "#0072B2" # blue - single-agent environment
COLOR_SOCIAL = "#D55E00" # vermillion - two-agent environment

def _plot_band(Onk, noise_levels, color, ax):
    q1, med, q3 = np.percentile(Onk, [5, 50, 95], axis=1)
    ax.fill_between(noise_levels, q1, q3, color=color, alpha=0.4)
    ax.plot(noise_levels, med, '-', color=color)
    ax.set_ylim(10, 2750)

def plot_convergence(NoiVar, noise_levels, ax, maxT=5000):
    kw = dict(noise_levels=noise_levels, ax=ax)
    _plot_band(NoiVar.sa_Tnk, color=COLOR_SINGLE, **kw)
    _plot_band(NoiVar.ta_Tnk, color=COLOR_SOCIAL, **kw)
    ax.set_yscale('log')
    ax.set_xlim(-0.05, max(noise_levels) + 0.05)
    ax.xaxis.set_minor_locator(MultipleLocator(0.2))
    ax.grid(True, which='major', axis='y', alpha=0.4)
    ax.set_xlabel('Observational uncertainty $\sigma$')

def plot_outperformance(NoiVar, noise_levels, ax):
    ci = collective_intelligence(NoiVar.ta_Rnki.mean(-1), NoiVar.sa_Rnki.mean(-1))
    # q1, med, q3 = np.percentile(ci, [5, 50, 95], axis=1)
    # ax.fill_between(noise_levels, q1, q3, color='purple', alpha=0.4)
    ax.plot(noise_levels, ci.mean(-1), 'k', color='k')
    ax.axhline(0, color='gray', lw=0.8, ls='--')
    ax.set_xlim(-0.05, max(noise_levels) + 0.05)
    ax.xaxis.set_minor_locator(MultipleLocator(0.2))
    ax.grid(True, which='major', axis='y', alpha=0.4)
    ax.set_xlabel('Observational uncertainty $\sigma$')
    ax.set_ylabel('Reward outperformance')

```

8.1 Learning rate α

We vary $\alpha \in \{0.125, 0.25, 0.5\}$ while keeping $\varepsilon = 10^{-6}$ (the baseline). The full 81-point noise grid is used. Results are cached via the existing hash-based infrastructure, so the $\alpha = 0.25$ baseline loads instantly and the two new values are computed once and stored.

```

def obtain_robustness_comparison_alpha(
    probabilityA, noise_levels, learning_rate,
    maxT=5000, n_initial=25,
    lowlevel_ddir='', highlevel_ddir=''):
    """Like obtain_noise_variation_comparison but with configurable
    ↳ learning_rate."""

```

```

agentparams = dict(learning_rates=learning_rate)
envparams    = dict(probabilityA=probabilityA, likelydistance=0.5)

# Two-agent
env = TwoAgentUncertainDecisionProblem(noiselevel=0.5, **envparams)
maei = LearningAgents(env, **agentparams)
initXkioa_ta = create_initial_lhs(maei, n_initial)
ta_Tnk, ta_Rnki, _ = obtain_noise_variation(
    TwoAgentUncertainDecisionProblem, noise_levels, envparams,
    LearningAgents, agentparams, initXkioa_ta, maxT=maxT,
    lowlevel_ddir=lowlevel_ddir, highlevel_ddir=highlevel_ddir)

# Single-agent
env = SingleAgentUncertainDecisionProblem(noiselevel=0.5, **envparams)
maei = LearningAgents(env, **agentparams)
initXkioa_sa = create_initial_lhs(maei, n_initial)
sa_Tnk, sa_Rnki, _ = obtain_noise_variation(
    SingleAgentUncertainDecisionProblem, noise_levels, envparams,
    LearningAgents, agentparams, initXkioa_sa, maxT=maxT,
    lowlevel_ddir=lowlevel_ddir, highlevel_ddir=highlevel_ddir)

NoiseVariation = namedtuple('NoiseVariation', ['ta_Tnk', 'sa_Tnk', 'ta_Rnki',
↪ 'sa_Rnki'])
return NoiseVariation(ta_Tnk=ta_Tnk, sa_Tnk=sa_Tnk, ta_Rnki=ta_Rnki,
↪ sa_Rnki=sa_Rnki)

```

```

alpha_results = {}
for alpha in alpha_values:
    print(f"\n=== alpha = {alpha} ===")
    alpha_results[alpha] = obtain_robustness_comparison_alpha(
        probabilityA=probabilityA,
        noise_levels=noise_levels,
        learning_rate=alpha,
        **ddirs)

```

```

=== alpha = 0.125 ===
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.125_initialX1284
↪ 2151645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.125_initialX7
↪ 794212447693859915_maxT5000_Noises7752999609125433264.npz

=== alpha = 0.25 ===
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initialX1284
↪ 151645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.25_initialX7
↪ 94212447693859915_maxT5000_Noises7752999609125433264.npz

```

```

=== alpha = 0.5 ===
Loading noise variation: ./_data/NOISEVARIATION_TwoAgentUncertainDecisionProblem_pr
↪ obabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.5_initialX128421
↪ 51645801287464_maxT5000_Noises7752999609125433264.npz
Loading noise variation: ./_data/NOISEVARIATION_SingleAgentUncertainDecisionProblem
↪ _probabilityA0.58_likelydistance0.5_LearningAgents_learning_rates0.5_initialX779
↪ 4212447693859915_maxT5000_Noises7752999609125433264.npz

fsf = 0.9
fig, axes = plt.subplots(2, 3, figsize=(fsf*13, fsf*6), sharey='row', sharex=True,
                        gridspec_kw=dict(hspace=0.08, wspace=0.1))

for col, alpha in enumerate(alpha_values):
    NoiVar = alpha_results[alpha]

    # Top: outperformance
    ax_top = axes[0, col]
    plot_outperformance(NoiVar, noise_levels, ax_top)
    ax_top.set_xlabel('')
    label = f'Learning rate  $\alpha = \{alpha\}$ '
    if alpha == 0.25:
        label += ' (baseline)'
    ax_top.set_title(label)

    # Bottom: convergence steps
    ax_bot = axes[1, col]
    plot_convergence(NoiVar, noise_levels, ax_bot, maxT=5000)

# y-labels only on left column
for row, ylabel in enumerate(['Reward outperformance', 'Convergence steps']):
    axes[row, 0].set_ylabel(ylabel)
    for col in range(1, 3):
        axes[row, col].set_ylabel('')

# x-labels only on bottom row
for col in range(3):
    axes[0, col].set_xlabel('')

legend_elements = [
    Line2D([0], [0], color=COLOR_SINGLE, lw=2, label='Single agent'),
    Line2D([0], [0], color=COLOR_SOCIAL, lw=2, label='Two agents')]
axes[1, 0].legend(handles=legend_elements, loc='lower right')

plt.savefig('./_figs/Robustness_LearningRate.png', dpi=300, bbox_inches='tight')

```

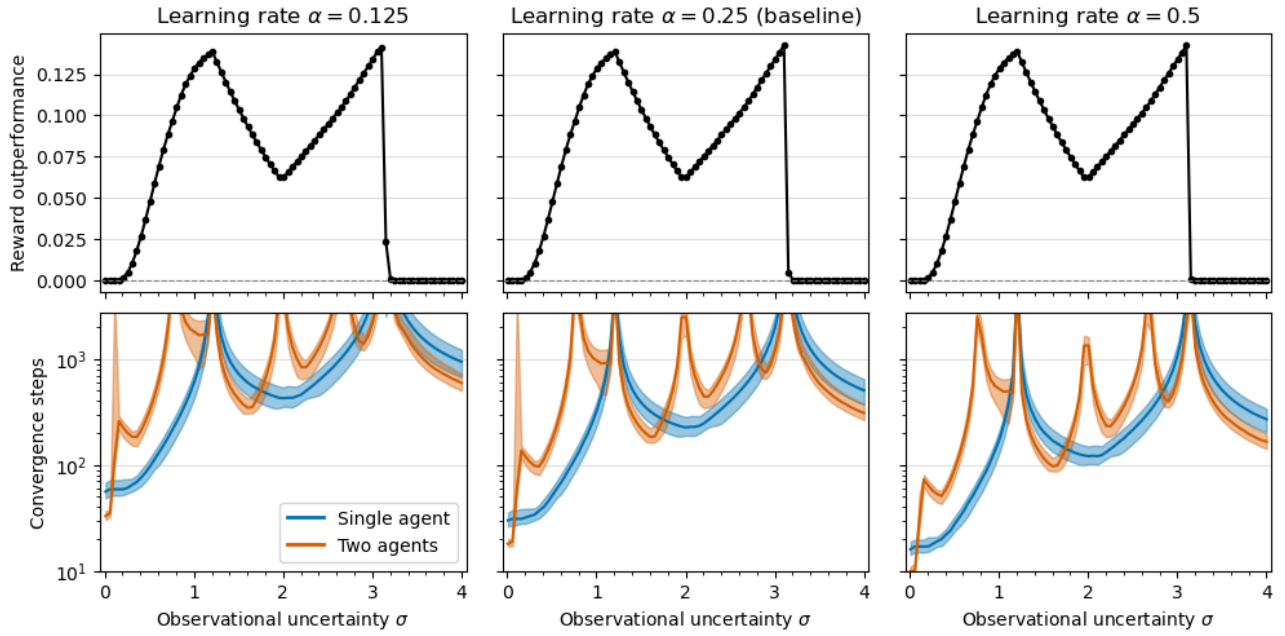



Figure 8.1: Robustness to learning rate α . Top row: reward outperformance shape (purple, median ± 5 –95th percentile). Bottom row: convergence timescales for single-agent (blue) and two-agent (red) models. State uncertainty $p_B = 0.42$, model-based agents, $\epsilon = 10^{-6}$.

8.2 Convergence threshold ε

We vary $\varepsilon \in \{10^{-5}, 10^{-6}, 10^{-7}\}$ while keeping $\alpha = 0.25$ (the baseline). The baseline $\varepsilon = 10^{-6}$ corresponds to `tolerance=10e-7` in `compute_trajectories`.

Because ε is not part of the existing cache-key scheme, we call `compute_trajectories` directly and use the coarser 21-point noise grid to keep computation time manageable. Results are **not** cached and will recompute on each notebook run.

```
def compute_convergence_with_tolerance(
    noise_levels, probabilityA, tolerance,
    learning_rate=0.25, maxT=5000, n_initial=10):
    """Compute convergence times and final rewards for both agent types with a given
    ↪ tolerance.
    Initial conditions are fixed (seed=42) and reused across noise levels."""
    agentparams = dict(learning_rates=learning_rate)
    envparams = dict(probabilityA=probabilityA, likelydistance=0.5)

    results = {}
    for name, envclass in [('two', TwoAgentUncertainDecisionProblem),
                           ('single', SingleAgentUncertainDecisionProblem)]:
        # Build initial conditions once from a reference environment
        env_ref = envclass(noiselevel=0.5, **envparams)
        maei_ref = LearningAgents(env_ref, **agentparams)
        initXkioa = create_inital_lhs(maei_ref, n_initial)

        conv_times = []
        rewards = []
        for ni, noise in enumerate(noise_levels):
```



```

        print(f"\r [{name}] noise {ni+1}/{len(noise_levels)}", end='')
        env = envclass(noiselevel=noise, **envparams)
        maei = LearningAgents(env, **agentparams)
        trjs, _ = compute_trajectories(maei, initXkioa, maxT=maxT,
↪ tolerance=tolerance)
        conv_times.append([len(t) for t in trjs])
        rewards.append(final_rewards_Rki(maei, trjs))
        print()
        results[name] = dict(Tnk=np.array(conv_times), Rnki=np.array(rewards))

NoiseVariation = namedtuple('NoiseVariation', ['ta_Tnk', 'sa_Tnk', 'ta_Rnki',
↪ 'sa_Rnki'])
return NoiseVariation(
    ta_Tnk=results['two']['Tnk'],    sa_Tnk=results['single']['Tnk'],
    ta_Rnki=results['two']['Rnki'],  sa_Rnki=results['single']['Rnki'])

```

```

epsilon_results = {}
for eps in epsilon_values:
    print(f"\n=== epsilon = {eps:.0e} ===")
    epsilon_results[eps] = compute_convergence_with_tolerance(
        noise_levels=noise_levels_coarse,
        probabilityA=probabilityA,
        tolerance=eps)

```

```

fsf = 0.85
fig, axes = plt.subplots(2, 3, figsize=(fsf*13, fsf*6), sharey='row', sharex=True,
                        gridspec_kw=dict(hspace=0.08, wspace=0.1))

for col, eps in enumerate(epsilon_values):
    NoiVar = epsilon_results[eps]

    # Top: outperformance
    ax_top = axes[0, col]
    plot_outperformance(NoiVar, noise_levels_coarse, ax_top)
    ax_top.set_xlabel('')
    label = f'Convergence threshold\n$\backslash$ varepsilon = {eps:.0e}$'
    if eps == 1e-6:
        label += ' (baseline)'
    ax_top.set_title(label)

    # Bottom: convergence steps
    ax_bot = axes[1, col]
    plot_convergence(NoiVar, noise_levels_coarse, ax_bot, maxT=5000)

# y-labels only on left column
for row, ylabel in enumerate(['Reward outperformance', 'Convergence steps']):
    axes[row, 0].set_ylabel(ylabel)
    for col in range(1, 3):
        axes[row, col].set_ylabel('')

# x-labels only on bottom row
for col in range(3):

```

```

axes[0, col].set_xlabel('')

legend_elements = [
    Line2D([0], [0], color=COLOR_SINGLE, lw=2, label='Single agent'),
    Line2D([0], [0], color=COLOR_SOCIAL, lw=2, label='Two agents')]
axes[1, 0].legend(handles=legend_elements, loc='lower right')

plt.savefig('./_figs/Robustness_ConvergenceThreshold.png', dpi=300,
            bbox_inches='tight')

```

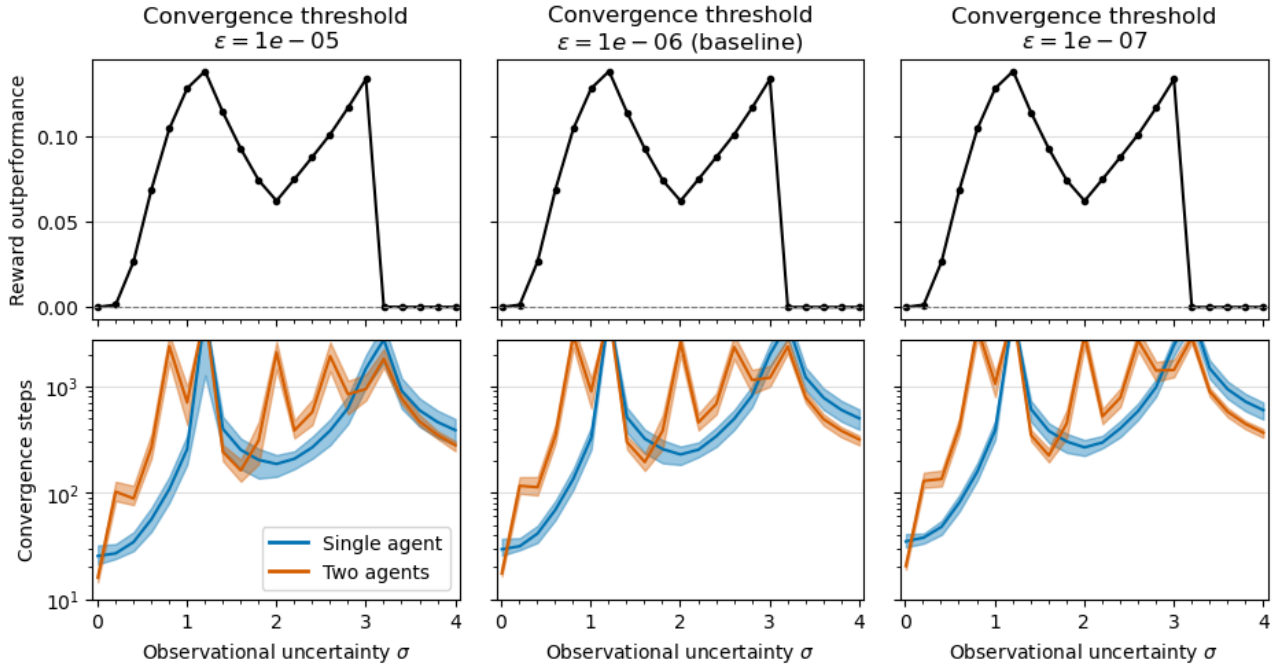


Figure 8.2: Robustness to convergence threshold ϵ . Top row: reward outperformance shape. Bottom row: convergence timescales. Coarser 21-point noise grid; 10 initial conditions; state uncertainty $p_B = 0.42$, model-based agents, $\alpha = 0.25$.

A Simulation scripts

Here, we provide the scripts to simulate the learning behavior of the agents in the model environment. We start by importing the required packages and defining some configurations.

```
#| default_exp SimulationScripts

#| export
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.cm as cm

from _code.UncertainDecisionProblem import \
    SingleAgentUncertainDecisionProblem, TwoAgentUncertainDecisionProblem
from _code.LearningDynamics import LearningAgents

from skopt.sampler import Lhs
from skopt.space import Space

import hashlib
from collections import namedtuple
```

To avoid issues with floating point arithmetic, we refine the `linspace` function from numpy.

```
#| export
def create_consistent_linspace(start, stop, number, decimals=8):
    return np.round(np.linspace(start, stop, number, dtype=np.float64), decimals)
```

A.1 Reward outperformance measure

We define the reward outperformance, a measure of collective intelligence, if you will, as the difference between the two-agent reward and the single-agent reward, normalized by the single-agent reward.

```
#| export
def collective_intelligence(multiagent_reward,
                           singleagent_reward):
    return (multiagent_reward - singleagent_reward) / singleagent_reward
```

A.2 Initial strategies

We use latin hypercube sampling to create the initial strategies.

```

#| export
def create_initial_lhs(maei, # multiagent environment interface
                      number, # number of initial policies to generate
                      iterations=1000, # number of iterations used for latin
↪ hypercube sampling
                      seed=42): # seed for random number generator
    """
    Returns near random samples of initial policies
    """
    assert maei.M == 2, 'Sampling for M>2 not straightforward'
    # https://www.egr.msu.edu/~kdeb/papers/c2018010.pdf
    # https://www.cs.cmu.edu/~nasmith/papers/smith+tromble.tr04.pdf

    eps = 10**(-6)
    space = Space(maei.N * maei.Q * (maei.M-1)*[(0.0+eps, 1.0-eps)])

    # generate latin hyper cubes
    lhs = Lhs(criterion="maximin", iterations=iterations)
    x = lhs.generate(space.dimensions, number, random_state=seed)
    x = np.array(x).reshape(number, maei.N, maei.Q, maei.M-1)

    # complete and normalize
    inits = np.zeros((number, maei.N, maei.Q, maei.M))
    inits[:, :, :, 0] = x[:, :, :, 0]
    inits[:, :, :, 1] = 1 - x[:, :, :, 0]

    return inits

```

Example latin hyper cube sampling

We test the latin hyper cube sample for 4 initial strategies at the single-agent decision problem. For each state s , we find exactly one strategy with $\frac{i}{4} < X_{0,s,0} \leq \frac{i+1}{4}$ for each $0 \leq i < 4$.

```

number = 4
env = SingleAgentUncertainDecisionProblem(noiselevel=0.5)
maei = LearningAgents(env, learning_rates=0.25, discount_factors=0.9)
Xkioa = create_initial_lhs(maei, number)

fig, axes = plt.subplots(1,4, figsize=(14,3))
def generate_colors(number): return cm.rainbow(np.linspace(0, 1, number))

for j, ax in enumerate(axes):
    probs = []
    for i in range(0, number):
        probs.append(Xkioa[i][0][j][0]) #agent 0 action 0 under observation 0
        ax.axhline(y=i/number, color='gray', linestyle='-', linewidth=0.5)

    ax.axhline(y=1.0, color='gray', linestyle='-', linewidth=0.5)
    ax.scatter(np.arange(number), probs, color=generate_colors(number),
               marker='o', s=50)
    ax.set_ylabel(f'X(s={j})')

```

```
plt.tight_layout()
```

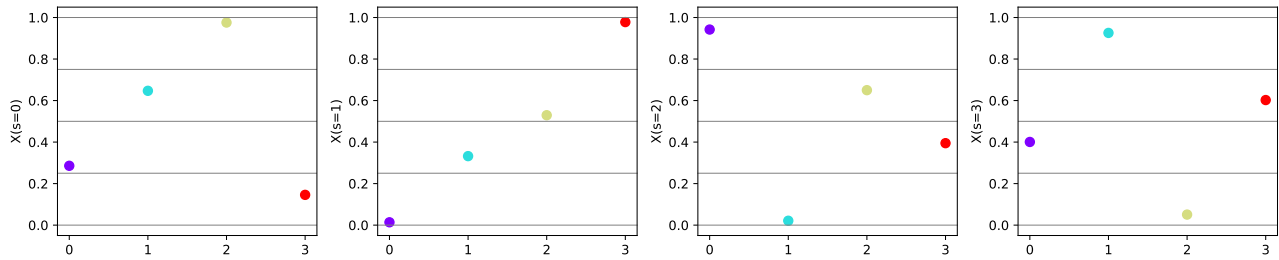


Figure A.1: Example of the initial strategies as latin hypercube samples.

A.3 Learning trajectories

Compute

First, we create a function to compute the policy trajectory for a given set of policies.

```
#| export
def compute_trajectories(maei,          # multiagent environment interface
                        inits,          # initial policies
                        maxT=1000,     # maximal number of learning steps
                        tolerance=10e-7): # tolerance for convergence
    """
    Returns array of policy trajectories for different initial policies and
    array of whether they converged
    """
    trjs = []; fprs = []
    leni = len(inits)

    for xi, x0 in enumerate(inits):
        print("\r [Computing policy trajectories]:",
              np.round(xi/leni, 4), end='')
        x = x0.copy()

        trj, fpr = maei.trajectory(x, Tmax=maxT, tolerance=tolerance)

        trjs.append(trj.astype(np.float32))
        fprs.append(fpr)

    print("")
    return np.array(trjs, dtype=object), np.array(fprs)
```

Testing the function,

```
number = 7
env = TwoAgentUncertainDecisionProblem(noiselevel=0.5)
# maei = POstratAC(env, learning_rates=0.25, discount_factors=0.9)
```

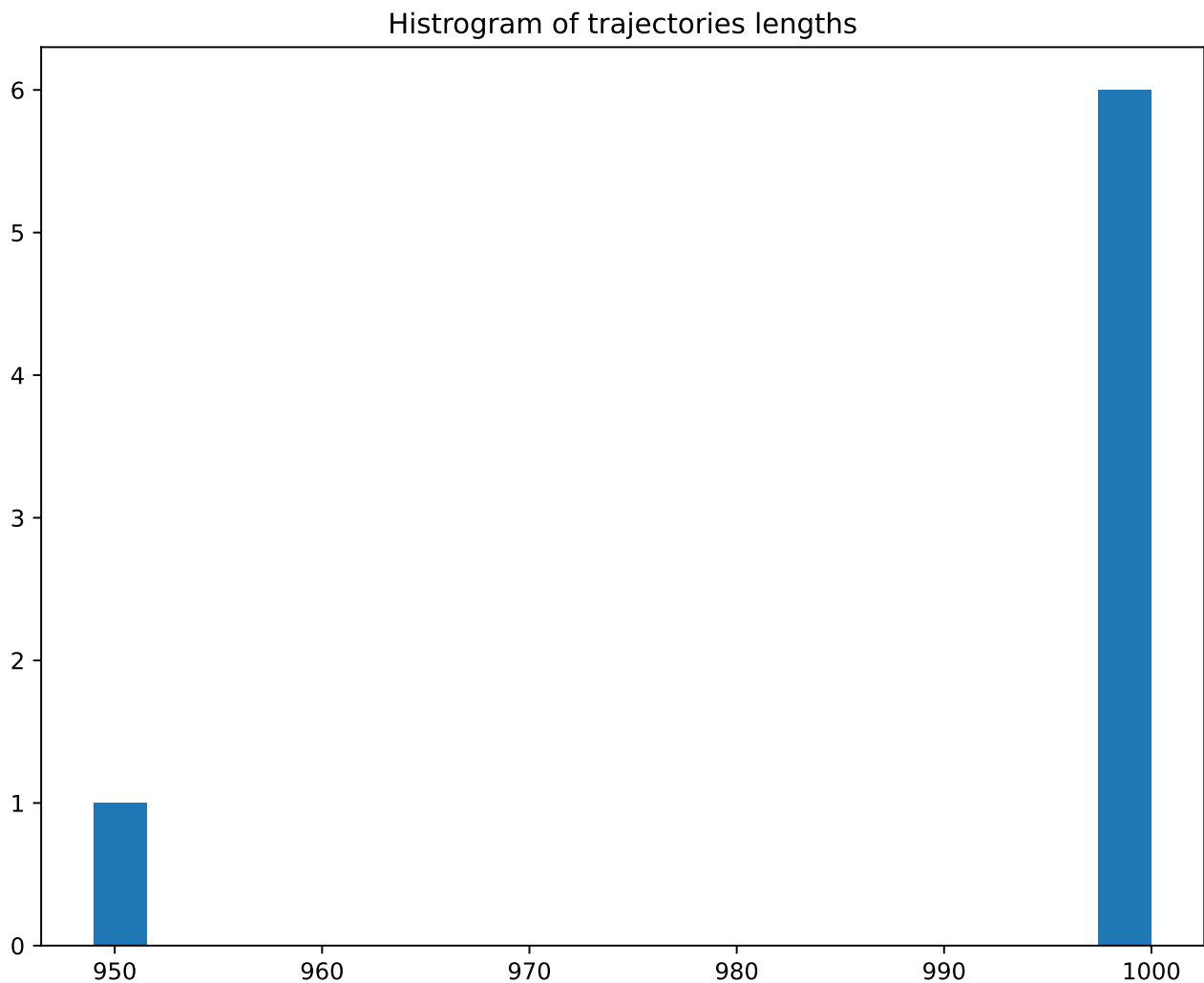



Figure A.2: Histogram of learning time steps required for a set of different initial strategies.

To save the computed runs to a re-identifiable file, we create a function `_transform_tensor_into_hash` which transforms an initial policy tensor into a hash.

```
#!/ export
def _transform_tensor_into_hash(tens):
    """Transform tens into a string for filename saving"""
    r = int(hashlib.sha512(str(tens).encode('utf-8')).hexdigest()[16], 16)
    return r
```

For example,

```
_transform_tensor_into_hash(Xkioa)
```

10980271089415640880

Obtain

We summarize these computation functions into a `obtain_trajectories` function.

```

#| export
def obtain_trajectories(maei, # multiagent environment interface
                       initXkioa, # initial policies
                       verbose=True, # indicates if explanations should be printed
                       maxT=5000, # maximal number of learning steps
                       ddir='data'): # directory where to save data
    """
    Loads or computes and saves learning trajectories.
    """
    fn = ddir + '/TRAJECTORIES_' + maei.id() + '_' + \
        + str(_transform_tensor_into_hash(initXkioa))
    fn += ".npz"

    try:
        dat = np.load(fn, allow_pickle=True)
        ddic = dict(zip((k for k in dat), (dat[k] for k in dat)))
        print("Loading ", fn) if verbose else None

    except:
        print("Computing ", fn) if verbose else None
        trjs, fprs = compute_trajectories(maei, initXkioa, maxT=maxT)
        check_run(trjs, fprs)
        # rtrajs = obtain_rewards(AEi, trajs)$

        ddic = dict(trjs=trjs, fprs=fprs)
        np.savez_compressed(fn, **ddic)
        dat = np.load(fn, allow_pickle=True)
        ddic = dict(zip((k for k in dat), (dat[k] for k in dat)))

    return ddic['trjs'], ddic['fprs']

```

Testing the function, first when there is prior file.

```
timeseries_Xktioa, fprs = obtain_trajectories(maei, Xkioa, maxT=1000, ddir='.')
```

Output hidden in PDF export.

When the file exists, it is not re-computed.

```
timeseries_Xktioa, fprs = obtain_trajectories(maei, Xkioa, maxT=1000, ddir='.')
```

```

Loading  ./TRAJECTORIES_TwoAgentUncertainDecisionProblem_0.5_0.5_1__jLearningAgents_
↪ _PartObs_[0.25 0.25]_10980271089415640880.npz

```

We delete the data, since this was only to test the function. In later use, we will give a more pronounced data directory.

```
!rm *.npz
```


Final strategies

We also want to study the final strategies that the agents learn.

```
#| export
def final_policies_Xkioa(Xktioa):
    return np.array([Xtioa[-1] for Xtioa in Xktioa])
```

Testing,

```
final_Xkioa = final_policies_Xkioa(timeseries_Xktioa)
final_Xkioa.shape
```

(7, 2, 13, 2)

Final rewards

Last, we create a function to compute the final rewards of a set of learning trajectories.

```
#| export
def final_rewards_Rki(maei, # multiagent environment interface
                     Xktioa): # learning trajectories
    """
    Returns the rewards `Rki` that agents receive at the end of the learning
    trajectories `Xktioa`.
    """
    if not maei.has_last_obsdist: # to avoid a small bug
        maei.obsdist(Xktioa[0][-1].astype(np.float32)) # with jax' just-in-time
    ↪ compilation

    Rki = []
    for Xtioa in Xktioa:
        Xioa = Xtioa[-1]
        Ri = maei.Ri(Xioa.astype(np.float32))
        Rki.append(Ri)

    return np.array(Rki)
```

Testing the function,

```
final_rewards_Rki(maei, timeseries_Xktioa)
```

```
array([[0.6862721 , 0.68627197],
       [0.6862698 , 0.68627214],
       [0.68627185, 0.68627137],
       [0.68627226, 0.68627185],
       [0.68627214, 0.68627197],
       [0.68627226, 0.6862722 ],
       [0.6862722 , 0.6862723 ]], dtype=float32)
```

A.4 Varying noise

Next, we create functions to compute the ensemble runs of the learning trajectories when varying the noise parameter.

Computing

```
#!/ export
def compute_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class
    agentparams,       # dictionary of agent parameters
    initialXkioa,      # initial policies, k indicates index of initial policy
    maxT=5000,         # maximal number of learning steps
    ddir='data'):      # directory to save and load data
    """Computes the learning trajectories for different noise levels.
        Returns the final rewards and convergence times."""

    convergence_times_Tnk = [] # n indicates noise index
    final_rewards_Rnki = []
    final_policies_Xnkioa = []

    for noise in noises:
        print(f" = = = {noise} = = = ")
        env = envclass(noiselevel=noise, **envparams)
        maei = agentclass(env, **agentparams)

        Xktioa, FPRk = obtain_trajectories(
            maei, initialXkioa, verbose=True, maxT=maxT, ddir=ddir)

        convergence_times_Tnk.append([len(Xtios) for Xtios in Xktioa])
        final_rewards_Rnki.append(final_rewards_Rki(maei, Xktioa))
        final_policies_Xnkioa.append(final_policies_Xkioa(Xktioa))

    return (np.array(convergence_times_Tnk),
            np.array(final_rewards_Rnki),
            np.array(final_policies_Xnkioa))
```

Testing the function with the single agent environment.

```
number_of_initial_policies_k = 4
agentclass = LearningAgents
agentparams = dict(learning_rates=0.25, discount_factors=0.9, use_prefactor=False)
envclass = SingleAgentUncertainDecisionProblem
envparams = dict(probabilityA=0.5, likelydistance=1)
env = envclass(noiselevel=0.5)
maei = agentclass(env, **agentparams)
```

```

initialXkioa = create_inital_lhs(maei, number_of_initial_policies_k)

noises = [0.001, 0.1, 1.0]

convergence_Tnk, final_Rnki, final_Xnkioa = compute_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class
    agentparams,       # dictionary of agent parameters
    initialXkioa,      # initial policies, k indicates index of initial policy
    ddir='.')

print()
assert convergence_Tnk.shape == (len(noises), number_of_initial_policies_k)
print("Shape of convergence times: ", convergence_Tnk.shape)
print()
assert final_Rnki.shape == (len(noises), number_of_initial_policies_k, maei.N)
print("Shape of final rewards: ", final_Rnki.shape)
print()
assert final_Xnkioa.shape == (len(noises), number_of_initial_policies_k, maei.N,
    ↪ maei.Q, maei.M)
print("Shape of final policies: ", final_Xnkioa.shape)

```

Output hidden in PDF export.

Testing the function with the two agent environment.

```

number_of_initial_policies_k = 4
agentclass = LearningAgents
agentparams = dict(learning_rates=0.25, discount_factors=0.9, use_prefactor=False)
envclass = TwoAgentUncertainDecisionProblem
envparams = dict(probabilityA=0.5, likelydistance=1)
env = envclass(noiselevel=0.5)
maei = agentclass(env, **agentparams)
initialXkioa = create_inital_lhs(maei, number_of_initial_policies_k)

noises = [0.001, 0.1, 1.0]

convergence_Tnk, final_Rnki, final_Xnkioa = compute_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class
    agentparams,       # dictionary of agent parameters
    initialXkioa,      # initial policies, k indicates index of initial policy
    ddir='.')

print()
assert convergence_Tnk.shape == (len(noises), number_of_initial_policies_k)
print("Shape of convergence times: ", convergence_Tnk.shape)
print()

```

```

assert final_Rnki.shape == (len(noises), number_of_initial_policies_k, maei.N)
print("Shape of final rewards: ", final_Rnki.shape)
print()
assert final_Xnkioa.shape == (len(noises), number_of_initial_policies_k, maei.N,
    ↪ maei.Q, maei.M)
print("Shape of final policies: ", final_Xnkioa.shape)

```

Output hidden in PDF export.

We delete the data, since this was only to test the function. In later use, we will give a more pronounced data directory.

```
!rm *.npz
```

Obtaining

First, we create a little helper function which transform a dictionary of parameters into a string for the file name.

```

#| export
def dict_to_string(paramdic):
    string = ''
    for k, v in paramdic.items():
        string += f'{k}-{v}_'
    return string[:-1]

```

For example, the environmtal parameters,

```
envparams
```

```
{'probabilityA': 0.5, 'likelydistance': 1}
```

will be transformed into a string as

```
dict_to_string(envparams)
```

```
'probabilityA0.5_likelydistance1'
```

We enrich the `obtain_noise_variation` function with a second data directory. One is to retrieve the underlying learning trajectories (`lowlevel_ddir`), the other is to save the ensemble results (`highlevel_ddir`). The latter is to be tracked in the project's git repository, the former not.

```

#| export
def obtain_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class

```

```

agentparams,      # dictionary of agent parameters
initialXkioa,     # initial policies, k indicates index of initial policy
maxT=5000,        # maximal number of learning steps
lowlevel_ddir='', # directory to save and load low-level data
highlevel_ddir='', # directory to save and load high-level data
):
    """Computes the learning trajectories for different noise levels.
    Returns the final rewards and convergence times."""

    # Create file name
    fn = highlevel_ddir + '/NOISEVARIATION_' + envclass.__name__ + '_' \
        + dict_to_string(envparams) \
        + '_' + agentclass.__name__ + '_' + dict_to_string(agentparams) \
        + '_' + "initiaX" + str(_transform_tensor_into_hash(initialXkioa)) \
        + '_' + "maxT" + str(maxT) \
        + '_' + "Noises" + str(_transform_tensor_into_hash(np.array(noises))) \
        + ".npz"

    try:
        dat = np.load(fn, allow_pickle=True)
        ddic = dict(zip((k for k in dat), (dat[k] for k in dat)))
        print("Loading noise variation:", fn)

    except:
        print("Computing noise variation:", fn)
        convergence_Tnk, final_Rnki, final_Xnkioa = compute_noise_variation(
            envclass,      # environment class
            noises,        # iterable of noise levels
            envparams,     # dictionary of environment parameters
            agentclass,    # agent class
            agentparams,   # dictionary of agent parameters
            initialXkioa,  # initial policies, k is index of initial policy
            maxT=maxT,     # maximal number of learning steps
            ddir=lowlevel_ddir)
        ddic = dict(convergence_times_Tnk=convergence_Tnk,
                    final_rewards_Rnki=final_Rnki,
                    final_policies_Xnkioa=final_Xnkioa)
        np.savez_compressed(fn, **ddic)
        dat = np.load(fn, allow_pickle=True)
        ddic = dict(zip((k for k in dat), (dat[k] for k in dat)))

    return (ddic['convergence_times_Tnk'],
            ddic['final_rewards_Rnki'],
            ddic['final_policies_Xnkioa'])

```

Testing the function with the two agent environment.

```
!mkdir _datatest
```

```

number_of_initial_policies_k = 4
agentclass = LearningAgents

```

```

agentparams = dict(learning_rates=0.25, discount_factors=0.9, use_prefactor=False)
envclass = TwoAgentUncertainDecisionProblem
envparams = dict(probabilityA=0.5, likelydistance=1)
env = envclass(noiselevel=0.5)
maei = agentclass(env, **agentparams)
initialXkioa = create_initial_lhs(maei, number_of_initial_policies_k)

noises = [0.001, 0.1, 1.0]

convergence_Tnk, final_Rnki, final_Xnkioa = obtain_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class
    agentparams,       # dictionary of agent parameters
    initialXkioa,      # initial policies, k indicates index of initial policy
    lowlevel_ddir='.',
    highlevel_ddir='_datatest')

print()
assert convergence_Tnk.shape == (len(noises), number_of_initial_policies_k)
print("Shape of convergence times: ", convergence_Tnk.shape)
print()
assert final_Rnki.shape == (len(noises), number_of_initial_policies_k, maei.N)
print("Shape of final rewards: ", final_Rnki.shape)
print()
assert final_Xnkioa.shape == (len(noises), number_of_initial_policies_k, maei.N,
    ↪ maei.Q, maei.M)
print("Shape of final policies: ", final_Xnkioa.shape)

```

Output hidden in PDF export.

Now, data is obtained much more quickly from disk:

```

convergence_Tnk, final_Rnki, final_Xnkioa = obtain_noise_variation(
    envclass,          # environment class
    noises,            # iterable of noise levels
    envparams,         # dictionary of environment parameters
    agentclass,        # agent class
    agentparams,       # dictionary of agent parameters
    initialXkioa,      # initial policies, k indicates index of initial policy
    lowlevel_ddir='.',
    highlevel_ddir='_datatest/')

print()
assert convergence_Tnk.shape == (len(noises), number_of_initial_policies_k)
print("Shape of convergence times: ", convergence_Tnk.shape)
print()
assert final_Rnki.shape == (len(noises), number_of_initial_policies_k, maei.N)
print("Shape of final rewards: ", final_Rnki.shape)
print()
assert final_Xnkioa.shape == (len(noises), number_of_initial_policies_k, maei.N,
    ↪ maei.Q, maei.M)

```

```
print("Shape of final policies: ", final_Xnkioa.shape)
```

Loading noise variation:

```
↪ _datatest//NOISEVARIATION_TwoAgentUncertainDecisionProblem_probabilityA0.5_like  
↪ lydistance1_LearningAgents_learning_rates0.25_discount_factors0.9_use_prefactor  
↪ False_initialX11824446990828570514_maxT5000_Noises8009511655927899652.npz
```

Shape of convergence times: (3, 4)

Shape of final rewards: (3, 4, 2)

Shape of final policies: (3, 4, 2, 13, 2)

Last, we delete the test data.

```
!rm *.npz
```

```
!rm -r _datatest
```

A.5 Noise variation comparison

Last, we create a function to obtain the comparison between single-agent and two-agent learning performance when varying the noise parameter.

```
#| export  
def obtain_noise_variation_comparison(lowlevel_ddir, highlevel_ddir,  
                                     probabilityA, noise_levels,  
                                     agentclass, maxT,  
                                     likelydistance=0.5,  
                                     number_of_initial_policies_k=25):  
  
    # Fixed parameters  
    agentparams = dict(learning_rates=0.25)  
    envparams = dict(probabilityA=probabilityA, likelydistance=likelydistance)  
  
    ## Two-agent model  
    envclass = TwoAgentUncertainDecisionProblem  
    env = envclass(noiselevel=0.5, **envparams)  
    maei = agentclass(env, **agentparams)  
  
    if number_of_initial_policies_k == 1:  
        initialXkios = np.array([maei.zero_intelligence_policy()])  
    else:  
        initialXkios = create_initlal_lhs(maei, number_of_initial_policies_k)  
  
    twoagent_convergence_Tnk, twoagent_final_Rnki, twoagent_Xnkioa =\  
        obtain_noise_variation(  
            envclass,          # environment class  
            noise_levels,      # iterable of noise levels  
            envparams,         # dictionary of environment parameters  
            agentclass,        # agent class
```

```

agentparams,      # dictionary of agent parameters
initialXkios,     # initial policies, k indicates index of initial policy
maxT=maxT,        # maximal number of learning steps
lowlevel_ddir=lowlevel_ddir, highlevel_ddir=highlevel_ddir)

# Single-agent model
envclass = SingleAgentUncertainDecisionProblem
env = envclass(noiselevel=0.5, **envparams)
maei = agentclass(env, **agentparams)

if number_of_initial_policies_k == 1:
    initialXkios = np.array([maei.zero_intelligence_policy()])
else:
    initialXkios = create_initl_lhs(maei, number_of_initial_policies_k)

singleagent_convergence_Tnk, singleagent_final_Rnki, singleagent_Xnkioa =\
    obtain_noise_variation(
        envclass,      # environment class
        noise_levels,  # iterable of noise levels
        envparams,     # dictionary of environment parameters
        agentclass,    # agent class
        agentparams,   # dictionary of agent parameters
        initialXkios,  # initial policies, k indicates index of initial policy
        maxT=maxT,     # maximal number of learning steps
        lowlevel_ddir=lowlevel_ddir, highlevel_ddir=highlevel_ddir)

NoiseVariation = namedtuple('NoiseVariation',
                             ['ta_Tnk', 'ta_Rnki', 'ta_Xnkioa',
                              'sa_Tnk', 'sa_Rnki', 'sa_Xnkioa'])

return NoiseVariation(ta_Tnk=twoagent_convergence_Tnk,
                      ta_Rnki=twoagent_final_Rnki,
                      ta_Xnkioa=twoagent_Xnkioa,
                      sa_Tnk=singleagent_convergence_Tnk,
                      sa_Rnki=singleagent_final_Rnki,
                      sa_Xnkioa=singleagent_Xnkioa)

```

At the very end, we export all functions to the simulation scripts module.

```

import nbdev
nbdev.export.nb_export("j81_AUX_Simulation.ipynb", "_code")

```


B Obtain noise variation data

Here, we obtain the simulation data used in our results. We start by importing the required packages and defining some configurations.

```
#| default_exp ObtainNoiseVariationResults
```

```
#| export
# import numpy as np
from _code.LearningDynamics import LearningAgents, ModelFreeAgents
from _code.SimulationScripts import create_consistent_linspace
from _code.SimulationScripts import obtain_noise_variation_comparison
```

```
#| export
lowlevel_datadictionary = "/Users/wolf/Downloads/L2SocialData"
highlevel_datadictionary = "./_data"
```

```
#| export
noise_levels = create_consistent_linspace(0.01, 4.0, 81)
noise_levels
```

```
array([0.01      , 0.059875, 0.10975 , 0.159625, 0.2095   , 0.259375,
       0.30925 , 0.359125, 0.409   , 0.458875, 0.50875 , 0.558625,
       0.6085  , 0.658375, 0.70825 , 0.758125, 0.808   , 0.857875,
       0.90775 , 0.957625, 1.0075  , 1.057375, 1.10725 , 1.157125,
       1.207   , 1.256875, 1.30675 , 1.356625, 1.4065  , 1.456375,
       1.50625 , 1.556125, 1.606   , 1.655875, 1.70575 , 1.755625,
       1.8055  , 1.855375, 1.90525 , 1.955125, 2.005   , 2.054875,
       2.10475 , 2.154625, 2.2045  , 2.254375, 2.30425 , 2.354125,
       2.404   , 2.453875, 2.50375 , 2.553625, 2.6035  , 2.653375,
       2.70325 , 2.753125, 2.803   , 2.852875, 2.90275 , 2.952625,
       3.0025  , 3.052375, 3.10225 , 3.152125, 3.202   , 3.251875,
       3.30175 , 3.351625, 3.4015  , 3.451375, 3.50125 , 3.551125,
       3.601   , 3.650875, 3.70075 , 3.750625, 3.8005  , 3.850375,
       3.90025 , 3.950125, 4.       ])
```

```
#| export
ddirs = dict(lowlevel_ddir=lowlevel_datadictionary,
             highlevel_ddir=highlevel_datadictionary)

mobaparams = dict(agentclass=LearningAgents, maxT=5000,
  ↳ number_of_initial_policies_k=25)
mofeparams = dict(agentclass=ModelFreeAgents, maxT=250000,
  ↳ number_of_initial_policies_k=10)
```

Obtain data

Then, we create a small summary function to obtain the simulation results for different noise levels and state uncertainties.

```
#| export
def obtain_all_noise_variation_comparisons():
    NoiVar_MBA_pAhigh = obtain_noise_variation_comparison(
        probabilityA=0.5,
        noise_levels=noise_levels,
        **mobaparams, **ddirs)

    NoiVar_MFA_pAhigh = obtain_noise_variation_comparison(
        probabilityA=0.5,
        noise_levels=noise_levels,
        **mofeparams, **ddirs)

    NoiVar_MBA_pAmid = obtain_noise_variation_comparison(
        probabilityA=0.58,
        noise_levels=noise_levels,
        **mobaparams, **ddirs)

    NoiVar_MFA_pAmid = obtain_noise_variation_comparison(
        probabilityA=0.58,
        noise_levels=noise_levels,
        **mofeparams, **ddirs)

    NoiVar_MBA_pAlow = obtain_noise_variation_comparison(
        probabilityA=0.7,
        noise_levels=noise_levels,
        **mobaparams, **ddirs)

    NoiVar_MFA_pAlow = obtain_noise_variation_comparison(
        probabilityA=0.7,
        noise_levels=noise_levels,
        **mofeparams, **ddirs)

    return (NoiVar_MBA_pAhigh, NoiVar_MFA_pAhigh,
            NoiVar_MBA_pAmid, NoiVar_MFA_pAmid,
            NoiVar_MBA_pAlow, NoiVar_MFA_pAlow)
```

Finally, we export everything we need from this notebook chapter into its own Python module for use in the results chapters.

```
import nbdev
nbdev.export.nb_export("j82_AUX_ObtainResultsData.ipynb", "_code")
```